

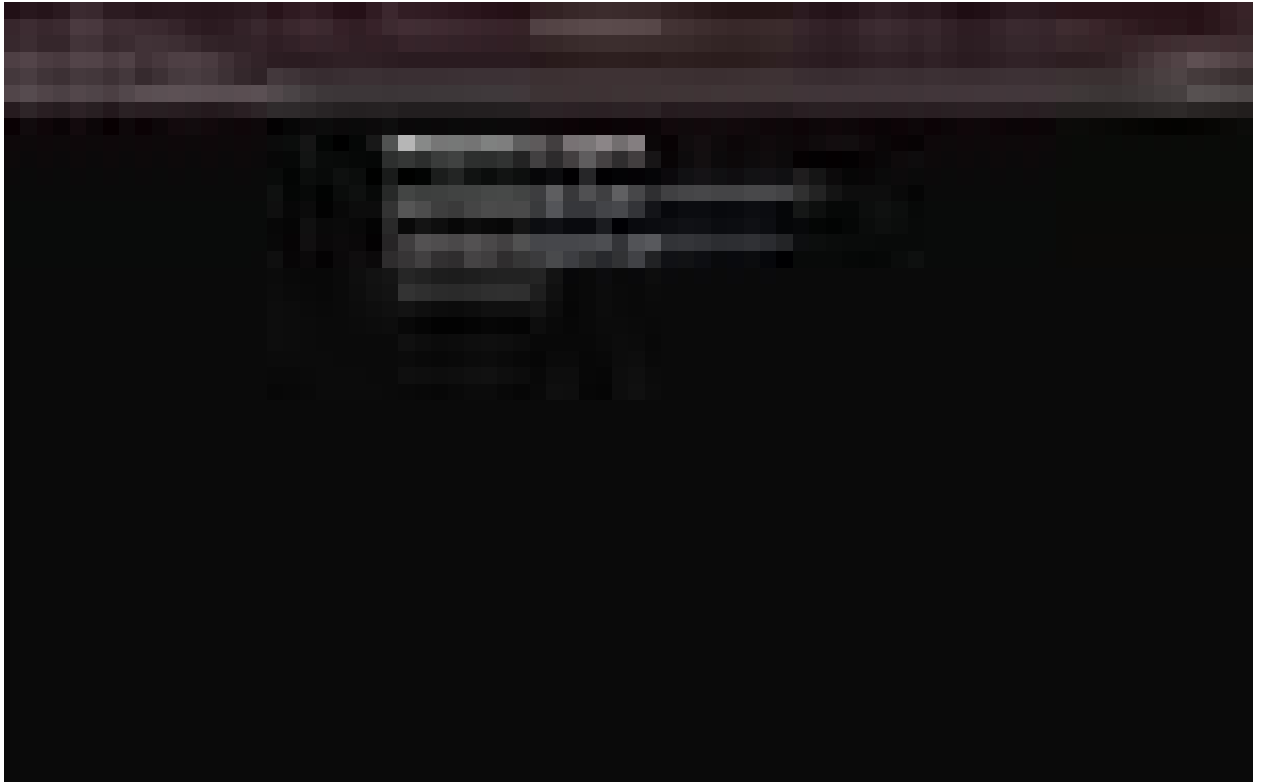
DevOps and Containerization Lab Report

Khushi Yadav

Contents

1 Experiment 1	15
1.0.1 Note:- As I am a mac-user I have performed the experiment on Someone else WIN-DOWS system.	15
1.1 Comparison of VMs and containers using Ubuntu and Nginx	15
1.2 Objective	15
1.3 Theory	15
1.3.1 Step 1: Install VirtualBox and vagrant.	16
1.3.2 Step 2: Install ubuntu	16
1.3.3 Step 3: start the VM	17
1.3.4 Step 4: Access the VM	18
1.3.5 Step 5: Verifying.	18
1.3.6 Step 6: Destroying and halting VMs	18
1.3.7 Step 7: Checking wsl	18
1.3.8 Step 8: Configuring	20
1.3.9 Step 9: Install Docker inside WSL	21
1.3.10 Step 10: Verify Docker Installation	21
1.3.11 Step 11: Install Nginx in Container	22
1.3.12 Step 12: Compare VM and Container Performance	22
1.4 Result	23
1.5 Conclusion	23
2 Experiment 2: Docker Installation, Configuration, and Running Images	23
2.1 Objective	23
2.2 Commands Executed	24
2.2.1 1. Docker Installation & Configuration Check	24
2.2.2 2. Pull Docker Image	24
2.2.3 3. Run Docker Container with Port Mapping	24
2.2.4 4. Container Lifecycle Management	26
2.2.5 5. Remove Docker Image	26
2.2.6 Result	26
3 Experiment 3: Deploying NGINX Using Different Base Images	26
3.1 Objective	26
4 Prerequisites	26
5 Part 1: Deploy NGINX Using Official Image	27
5.1 Step 1: Pull Image	27
5.2 Step 2: Run Container	27
5.3 Step 3: Verify	27

5.4

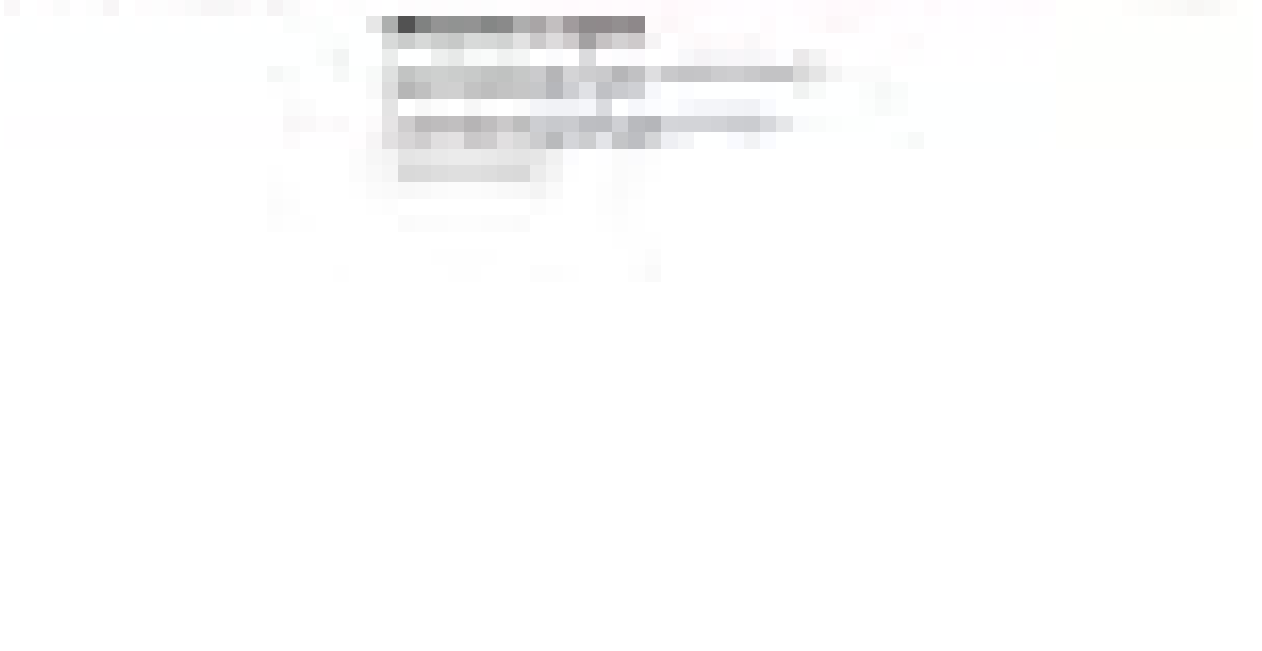


28

6	Part 2: Custom NGINX Using Ubuntu Base Image	28
6.1	Step 1: Create Dockerfile	28
6.2	Step 2: Build Image	28
6.3	Step 3: Run Container	28



6.4



29

29

7.4

30

31

8.2.1	Checked Container	31
8.2.2	Entered Container	31
8.2.3	Checked Default Config	31
9	Fix Applied	32
9.0.1	Created Web Directory	32
9.0.2	Modified Configuration	32
9.0.3	Tested Configuration	32
9.0.4	Reloaded NGINX	32
		
		
		
		
		
		
9.1		34
10	Image Size Comparison	34
10.1	Results:	34

11 Observations	35
12 Conclusion	35
13 Cleanup Commands	36
14 Key Learnings	36
15 Experiment 4 – Docker Essentials	36
15.1 Aim	36
16 Part 1 – Containerizing a Flask Application	36
16.1 Step 1: Create Project Directory	36
16.2 Step 2: Create Flask Application (app.py)	36
16.3 Step 3: Create requirements.txt	37
16.4 Step 4: Create Dockerfile	37
16.5 Step 5: Build Docker Image	37
16.6 Step 6: Run Container	37
16.7 Step 7: Test Health Endpoint	39
16.8 Step 8: View Logs	39

16.9	<pre>(base) khushi@khushi-MacBook-Air-2 my-flask-app % docker logs flask-container * Serving Flask app 'app' * Debug mode: off WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead. * Running on all addresses (0.0.0.0) * Running on http://127.0.0.1:5000 * Running on http://172.17.0.2:5000 Press CTRL+C to quit 192.168.65.1 - - [28/Feb/2025 03:37:34] "GET / HTTP/1.1" 200 - 192.168.65.1 - - [28/Feb/2025 03:37:34] "GET /favicon.ico HTTP/1.1" 404 - 192.168.65.1 - - [28/Feb/2025 03:39:42] "GET /health HTTP/1.1" 200 - (base) khushi@khushi-MacBook-Air-2 my-flask-app % docker stop flask-container flask-container (base) khushi@khushi-MacBook-Air-2 my-flask-app % docker ps CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES (base) khushi@khushi-MacBook-Air-2 my-flask-app % docker start flask-container flask-container (base) khushi@khushi-MacBook-Air-2 my-flask-app % docker ps CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES 837f12f03b my-flask-app "python app.py" 4 minutes ago Up 8 seconds 0.0.0.0:5001->5000/tcp, :::5001->5000/tcp flask-container</pre>	39
16.10	Step 9: Stop, Start and Remove Container	39
16.11	<pre>(base) khushi@khushi-MacBook-Air-2 my-flask-app % docker stop flask-container flask-container (base) khushi@khushi-MacBook-Air-2 my-flask-app % docker rm flask-container flask-container (base) khushi@khushi-MacBook-Air-2 my-flask-app % docker ps -a CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES 836e9d872ac7 ubuntu "/bin/bash" 2 days ago Exited (255) 21 minutes ago class 3e08f8611965 ubuntu "/bin/bash" 2 days ago Exited (0) 2 days ago keen_cerf 5c0f7423bd38 ubuntu "/bin/bash" 2 days ago Exited (0) 2 days ago laughing_brahmasuta 8a1ae74c6986 ubuntu "/bin/bash" 2 days ago Exited (0) 2 days ago lucia_loveritt 66f0963edc4e ubuntu "/bin/bash" 2 days ago Exited (0) 2 days ago keen_wormiak 6e0903be3177 ubuntu:firstimage:final "/bin/bash" 3 weeks ago Exited (127) 3 weeks ago nostalgic_bobir 475d8d180c38 ubuntu "/bin/bash" 3 weeks ago Exited (0) 3 weeks ago bold_sawashi 3c8f4ac2ae57 ubuntu "/bin/bash" 3 weeks ago Exited (0) 3 weeks ago blissful_tauusia 579f17926a6a ubuntu "/bin/bash" 4 weeks ago Exited (255) 11 days ago nifty_robertort 2e0943bb6237 05813aedc15f "/hello" 4 weeks ago Exited (0) 4 weeks ago tmatioo_acosnaciamaya (base) khushi@khushi-MacBook-Air-2 my-flask-app % docker images IMAGE ID DISK USAGE CONTENT SIZE EXTRA my-flask-app:latest 7c8fa35810ec 220MB 49.1MB ubuntu:firstimage:final f6199a7416fa 1.31GB 325MB 0 ubuntu:latest c51d0a651b38 143MB 30.0MB 0 (base) khushi@khushi-MacBook-Air-2 my-flask-app % username/repository:tag zsh: no such file or directory: username/repository:tag</pre>	40
17	Part 2 – Docker Hub Operations	40
17.1	Step 10: Tag Image	40
17.2	Step 11: Login to Docker Hub	40
17.3	Step 12: Push Image	41
17.4	Step 13: Remove Local Image	41
17.5	Step 14: Pull Image from Docker Hub	41
17.6	Step 15: Run Pulled Image	41
18	Errors and Issues Faced	42
18.1	-> Docker Daemon Not RunningAt 9 AM	42
18.2	-> Port 5000 Already in Use	42
18.3	-> Container Name Already in Use	42
19	Results	43
20	Conclusion	43
21	Experiment 5:	44
22	Docker — Volumes, Environment Variables, Monitoring & Networks	44
22.1	Table of Contents	44
22.2	Overview	44
22.3	Part 1: Docker Volumes	44
22.3.1	Lab 1: Understanding Data Persistence	44

```

Last login: Fri Mar 15 21:39:36 on console
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker ps
Cannot connect to the Docker daemon at unix:///Users/khushiyadaw/.docker/run/docker.sock. Is the docker daemon running?
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED        STATUS        PORTS                    NAMES
9d13094279e        wordpress:latest   "docker-entrypoint.s..." 2 weeks ago   Up 12 seconds  80/tcp                   docker-compose-demo-wordpress-1
6ed19640279e        wordpress:latest   "docker-entrypoint.s..." 2 weeks ago   Up 12 seconds   80/tcp                   docker-compose-demo-wordpress-2
332d711e19bd        mysql:8.0          "docker-entrypoint.s..." 2 weeks ago   Up 12 seconds   3306/tcp, 33060/tcp      mysql
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker run -it --name test-container ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
06a48bbf6b60: Pull complete
9c2a2ec78563: Download complete
Digest: sha256:d1e2e52075c5a139d51a1981ff95f84315c0fdce203eah28977ce93eff4f9
Status: Downloaded newer image for ubuntu:latest
root@710186bdea76:/# mkdir /data && echo "Hello World" > /data/message.txt && cat /data/message.txt
Hello World
root@710186bdea76:/# exit
exit

```

```

(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker start test-container
test-container
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker exec test-container cat /data/message.txt
Hello World
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker rm test-container
Error response from daemon: cannot remove container "test-container": container is running; stop the container before removing or force remove
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker stop test-container
test-container
test-container
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker run -it --name test-container ubuntu /bin/bash
root@6c8eece99804:/# cat /data/message.txt
cat: /data/message.txt: No such file or directory
root@6c8eece99804:/# exit
exit

```

22.5.1 Lab 2: Volume Types 46

```

(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % mkdir ~/myapp-data
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker run -d -v ~/myapp-data:/app/data --name web3 nginx
87c7b6e8fd62a774f1d2494c8dd817c59271b08471ed2bb93288c5f9195e2952
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % echo "From Host" > ~/myapp-data/host-file.txt
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker exec web3 cat /app/data/host-file.txt
From Host
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker run -d \
--name mysql-db \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
87af7fe815acd7fd2b2d23b8adaf58ca337c9bb92896af1375c742b50caab69a

```

22.6.1 Lab 3: Practical Volume Examples 47

```

(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker run -d \
--name mysql-db \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
87af7fe815acd7fd2b2d23b8adaf58ca337c9bb92896af1375c742b50caab69a
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED        STATUS        PORTS                    NAMES
87af7fe815acd        mysql:8.0          "docker-entrypoint.s..." 22 seconds ago Up 22 seconds   3306/tcp, 33060/tcp      mysql-db
87c7b6e8fd62        nginx              "nginx -g 'daemon off..." About a minute ago Up About a minute   80/tcp                   web3
6d2907f92ef2        nginx              "nginx -g 'daemon off..." 3 minutes ago   Up 3 minutes     80/tcp                   web2
87f634732c70        nginx              "nginx -g 'daemon off..." 4 minutes ago   Up 4 minutes     80/tcp                   web1
9d13094279e        wordpress:latest   "docker-entrypoint.s..." 2 weeks ago   Up 15 minutes   80/tcp                   docker-compose-demo-wordpress-1
6ed19640279e        wordpress:latest   "docker-entrypoint.s..." 2 weeks ago   Up 15 minutes   80/tcp                   docker-compose-demo-wordpress-2
332d711e19bd        mysql:8.0          "docker-entrypoint.s..." 2 weeks ago   Up 15 minutes   3306/tcp, 33060/tcp      mysql
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED        STATUS        PORTS                    NAMES
87af7fe815acd        mysql:8.0          "docker-entrypoint.s..." 2 minutes ago Up 2 minutes     3306/tcp, 33060/tcp      mysql-db
87c7b6e8fd62        nginx              "nginx -g 'daemon off..." 3 minutes ago Up 3 minutes     80/tcp                   web3
6d2907f92ef2        nginx              "nginx -g 'daemon off..." 5 minutes ago Up 5 minutes     80/tcp                   web2
87f634732c70        nginx              "nginx -g 'daemon off..." 7 minutes ago Up 7 minutes     80/tcp                   web1
9d13094279e        wordpress:latest   "docker-entrypoint.s..." 2 weeks ago Up 13 minutes   80/tcp                   docker-compose-demo-wordpress-1
6ed19640279e        wordpress:latest   "docker-entrypoint.s..." 2 weeks ago Up 13 minutes   80/tcp                   docker-compose-demo-wordpress-2
332d711e19bd        mysql:8.0          "docker-entrypoint.s..." 2 weeks ago Up 13 minutes   3306/tcp, 33060/tcp      mysql
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker stop mysql-db
mysql-db
mysql-db
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker run -d \
--name new-mysql \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
6c8a21564ea319f90665529c79851be1856b6c52948ec448274ef28eab99c3
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED        STATUS        PORTS                    NAMES
6c8a21564ea3        mysql:8.0          "docker-entrypoint.s..." 7 seconds ago Up 7 seconds     3306/tcp, 33060/tcp      new-mysql
87c7b6e8fd62        nginx              "nginx -g 'daemon off..." 4 minutes ago Up 4 minutes     80/tcp                   web3
6d2907f92ef2        nginx              "nginx -g 'daemon off..." 6 minutes ago Up 6 minutes     80/tcp                   web2
87f634732c70        nginx              "nginx -g 'daemon off..." 7 minutes ago Up 7 minutes     80/tcp                   web1
9d13094279e        wordpress:latest   "docker-entrypoint.s..." 2 weeks ago Up 14 minutes   80/tcp                   docker-compose-demo-wordpress-1
6ed19640279e        wordpress:latest   "docker-entrypoint.s..." 2 weeks ago Up 14 minutes   80/tcp                   docker-compose-demo-wordpress-2
332d711e19bd        mysql:8.0          "docker-entrypoint.s..." 2 weeks ago Up 14 minutes   3306/tcp, 33060/tcp      mysql

```



```

(base) khushiyadav@Khushis-MacBook-Air-2 ~ % mkdir ~/nginx-config
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % echo 'server {
    listen 80;
    server_name localhost;
    location / {
        return 200 "Hello from mounted config!";
    }
}' > ~/nginx-config/nginx.conf
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker run -d \
    --name nginx-custom \
    -p 8080:80 \
    -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
    nginx
1a8a47eb558c657686ad247e0e1652a92359d4dba7b5044dcd12f4f1d4b6a66e
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % curl http://localhost:8080
Hello from mounted config!%

```

22.8

22.8.1 Lab 4: Volume Management Commands 49

```

khushiyadav@Khushis-MacBook-Air-2 ~ % docker volume ls
DRIVER      VOLUME NAME
local       8656cd6c89f184e4e895d0b2469679b92986d870d95d17f5af70bfac78a5f777
local       docker-compose-demo_mysql_data
local       docker-compose-demo_wp_content
local       mydata
local       mysql-data
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker volume create app-volume
app-volume
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker volume inspect app-volume
[
  {
    "CreatedAt": "2026-03-14T17:48:24Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
    "Name": "app-volume",
    "Options": null,
    "Scope": "local"
  }
]
(base) khushiyadav@Khushis-MacBook-Air-2 ~ % docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B

```

22.9

22.10Part 2: Environment Variables 49

22.10.1 Lab 1: Setting Environment Variables 50


```

(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker run -d \
  --name app1 \
  -e VAR1=value1 \
  -e VAR2=value2 \
  -e VAR3=value3 \
  nginx
40eff99d012f40757151c10d9ab370f7d60c1a612c5fd99f7f015f02d6e0b7d8
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker exec app1 env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=40eff99d012f
VAR1=value1
VAR2=value2
VAR3=value3
NGINX_VERSION=1.29.6
NJS_VERSION=0.9.6
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
HOME=/root
22.11

```

```

(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % cd ~ && echo "DATABASE_HOST=localhost" > .env
echo "DATABASE_PORT=5432" >> .env
echo "API_KEY=secret123" >> .env
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % cat ~/.env
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker run -d \
  --name app2 \
  --env-file ~/.env \
  nginx
402c81004d27024f99a8c8fa24bf69c0783ab907726625c7f6ae8edaa3f87e60
(base) khushiyadaw@Khushis-MacBook-Air-2 ~ % docker exec app2 env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=402c81004d27
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
NGINX_VERSION=1.29.6
NJS_VERSION=0.9.6
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
HOME=/root
22.12

```

22.12.1 Lab 2: Flask App with Environment Variables 51

```

=> exporting to image
=> exporting layers
=> exporting manifest sha256:a84db6c89f44c1d729c2c3cc6b68f96e7b67d693d1988ac32fa574593f6815
=> exporting config sha256:26084874839871d55664b26c124d4fa48a6c53153e186c41c38443c8e0a8280
=> exporting attestation manifest sha256:1ab594731debf1e84871beaf0143d0f8da989ca913366a74dca4a1fa8875db
=> exporting manifest list sha256:ca08450ab0012098c1950977c3a0833889c73c10812521132f4440e5197880
=> naming to docker.io/library/flask-app:latest
=> unpacking to docker.io/library/flask-app:latest
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker run -d \
--name flask-app \
-p 5000:5000 \
-e DATABASE_HOST="prod-db.example.com" \
-e DEBUG="true" \
-e API_KEY="myapikey123" \
flask-app
59a0a605ef70c0f548f6a3a2a4cd32146df25f9f918f69462679cd5609cc
docker: Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:5000 -> 127.0.0.1:0: listen tcp 0.0.0.0:5000: bind: address already in use

Run 'docker run --help' for more information
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker rm flask-app
flask-app
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker run -d \
--name flask-app \
-p 5001:5000 \
-e DATABASE_HOST="prod-db.example.com" \
-e DEBUG="true" \
-e API_KEY="myapikey123" \
flask-app
c34ad32cf7324f355a65461d2c891bc95d3c5d18938f38f1ea2c0160a82e824
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker logs flask-app
+ Serving Flask app 'app'
+ Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
+ Running on all addresses (0.0.0.0)
+ Running on http://127.0.0.1:5000
+ Running on http://172.17.0.9:5000
Press CTRL-C to quit
+ Restarting with stat
+ Debugger is active!
+ Debugger PIN: 185-668-538
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % curl http://localhost:5001/config
{
  "db_host": "prod-db.example.com",
  "debug": true,
  "has_api_key": true
}

```

22.13

55

22.14Part 3: Docker Monitoring 55

22.14.1 Lab 1: Resource Metrics 55

```

(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker stats --no-stream
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O     BLOCK I/O     PIDS
e68ad12ef192   flask-app 0.23%     43.34MiB / 3.827GiB  1.33%     1.55kB / 792B  0B / 0B       2
480c81884627   app2      0.00%     7.588MiB / 3.827GiB  0.19%     1.46kB / 126B  0B / 12.3kB   9
48c7f938812f   app1      0.00%     7.414MiB / 3.827GiB  0.19%     1.59kB / 126B  0B / 12.3kB   9
1a8a47eb558c   nginx-cs100  0.00%     7.531MiB / 3.827GiB  0.19%     2.55kB / 816B  0B / 4.1kB     9
bc8a213646a3   new-mysql  0.01%     367.4MiB / 3.827GiB  9.38%     1.88kB / 126B  65.5kB / 15.3kB 37
8fc70a0d062   web3      0.00%     7.473MiB / 3.827GiB  0.19%     2.13kB / 126B  69.8kB / 12.3kB 9
1802997f93d12  web2      0.00%     7.508MiB / 3.827GiB  0.19%     2.26kB / 126B  0B / 12.3kB    9
8ff634732c70   web1      0.00%     9.613MiB / 3.827GiB  0.25%     2.68kB / 126B  2.37kB / 12.3kB 9
9447072ef71a   docker-compose-down-wordpress-1  0.01%     41.84MiB / 512MiB   8.01%     2kB / 126B    27.7kB / 4.1kB  6
6ed19a58270e   docker-compose-down-wordpress-2  0.01%     48.85MiB / 512MiB   7.58%     2kB / 126B    26.9kB / 4.1kB  6
33d7f15e19ed   mysql     0.72%     415.3MiB / 3.827GiB  10.59%     2kB / 126B    76.9kB / 17kB   37
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.NetIO}}"
NAME           CPU %    MEM USAGE / LIMIT   NET I/O
flask-app      0.23%    43.34MiB / 3.827GiB  1.55kB / 792B
app2           0.00%    7.588MiB / 3.827GiB  1.46kB / 126B
app1           0.00%    7.414MiB / 3.827GiB  1.59kB / 126B
nginx-cs100    0.00%    7.531MiB / 3.827GiB  2.55kB / 816B
new-mysql     0.01%    367.4MiB / 3.827GiB  1.88kB / 126B
web3           0.00%    7.473MiB / 3.827GiB  2.13kB / 126B
web2           0.00%    7.508MiB / 3.827GiB  2.26kB / 126B
web1           0.00%    9.613MiB / 3.827GiB  2.68kB / 126B
docker-compose-down-wordpress-1  0.01%    41.84MiB / 512MiB   2kB / 126B
docker-compose-down-wordpress-2  0.01%    48.85MiB / 512MiB   2kB / 126B
mysql          0.72%    415.3MiB / 3.827GiB  2kB / 126B

```

22.15

56

```

(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker top flask-app
CWD      PID      PPID     C        STIME    TTY      TOW      CMD
root     2448     2425     0        10:07    ?        00:00:00 python -u app.py
root     2468     2448     0        10:07    ?        00:00:00 /usr/local/bin/python /usr/bin/redis-server /var/run/redis/redis.sock

```

22.16

56

22.16.1 Lab 2: Log Monitoring 56

```

[base@localhost ~]$ docker logs flask-app
+ Serving Flask app 'app'
+ Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
+ Running on all addresses (0.0.0.0)
+ Running on http://127.0.0.1:5000
+ Running on http://172.17.0.9:5000
Press CTRL+C to quit
+ Restarting with stat
+ Debugger is active!
+ Debugger PIN: 180-068-030
182.168.65.1 - - [14/Mar/2016 18:08:23] "GET /config HTTP/1.1" 200 -
[base@localhost ~]$ docker logs -t flask-app
2016-03-14T18:07:55.646147832 + Serving Flask app 'app'
2016-03-14T18:07:55.646147832 + Debug mode: on
2016-03-14T18:07:55.647028402 WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
2016-03-14T18:07:55.647042952 + Running on all addresses (0.0.0.0)
2016-03-14T18:07:55.647043872 + Running on http://127.0.0.1:5000
2016-03-14T18:07:55.647044872 + Running on http://172.17.0.9:5000
2016-03-14T18:07:55.647045782 Press CTRL+C to quit
2016-03-14T18:07:55.647125872 + Restarting with stat
2016-03-14T18:07:55.647168862 + Debugger is active!
2016-03-14T18:07:55.647169282 + Debugger PIN: 180-068-030
2016-03-14T18:08:23.147050402 182.168.65.1 - - [14/Mar/2016 18:08:23] "GET /config HTTP/1.1" 200 -
[base@localhost ~]$ docker logs -t flask-app --tail 5 flask-app
Press CTRL+C to quit
+ Restarting with stat
+ Debugger is active!
+ Debugger PIN: 180-068-030
182.168.65.1 - - [14/Mar/2016 18:08:23] "GET /config HTTP/1.1" 200 -
[base@localhost ~]$ flask-app & curl http://localhost:5001/config
curl http://localhost:5001/config
{
  "db_host": "prod-db.example.com",
  "debug": true,
  "api_key": true

  "db_host": "prod-db.example.com",
  "debug": true,
  "api_key": true

  "db_host": "prod-db.example.com",
  "debug": true,
  "api_key": true
}
[base@localhost ~]$ flask-app & docker logs -f --tail 10 -t flask-app
2016-03-14T18:07:55.647043872 + Running on http://127.0.0.1:5000
2016-03-14T18:07:55.647044872 + Running on http://172.17.0.9:5000
2016-03-14T18:07:55.647045782 Press CTRL+C to quit
2016-03-14T18:07:55.647125872 + Restarting with stat
2016-03-14T18:07:55.647168862 + Debugger is active!
2016-03-14T18:07:55.647169282 + Debugger PIN: 180-068-030
2016-03-14T18:08:23.147050402 182.168.65.1 - - [14/Mar/2016 18:08:23] "GET /config HTTP/1.1" 200 -
2016-03-14T18:14:17.528671132 182.168.65.1 - - [14/Mar/2016 18:14:17] "GET /config HTTP/1.1" 200 -
2016-03-14T18:14:17.531564282 182.168.65.1 - - [14/Mar/2016 18:14:17] "GET /config HTTP/1.1" 200 -
2016-03-14T18:14:17.534887482 182.168.65.1 - - [14/Mar/2016 18:14:17] "GET /config HTTP/1.1" 200 -
^C

```

22.17

57

22.17.1 Lab 3: Container Inspection 57

```

[base@localhost ~]$ flask-app & docker inspect --format '{{.State.Stats}}' flask-app
{
  "blkio_stats": {
    "io_service_bytes_recursive": {
      "bytes": 0,
      "operation": 0,
      "value": 0
    },
    "io_serviced_recursive": {
      "bytes": 0,
      "operation": 0,
      "value": 0
    },
    "io_time_recursive": {
      "bytes": 0,
      "operation": 0,
      "value": 0
    },
    "io_wait_time_recursive": {
      "bytes": 0,
      "operation": 0,
      "value": 0
    },
    "io_merged_recursive": {
      "bytes": 0,
      "operation": 0,
      "value": 0
    },
    "io_time": 0,
    "io_wait_time": 0,
    "io_merged": 0,
    "io_serviced": 0
  },
  "cpu_stats": {
    "cpu_usage": {
      "usage_in_user": 0,
      "usage_stolen": 0
    },
    "throttled_time": 0
  },
  "memory_stats": {
    "memory_usage": {
      "memory_usage": 0,
      "memory_working_set": 0
    },
    "memory_swap": {
      "memory_swap": 0,
      "memory_swap_limit": 0
    },
    "memory_total": 0
  },
  "pids_stats": {
    "pids": 0
  },
  "time": 1440000000000.0
}
[base@localhost ~]$ docker system df

```

22.18

57

22.19 Part 4: Docker Networks 57

22.19.1 Lab 1: Network Types 58

```

0010 Cache 11 0 117.140 24.30K
(base) shushiyada@Khushis-MacBook-Air-2 flask-app % docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
f484b063b6a8        bridge              bridge              local
27941e90f3c5        docker-compose-demo_default bridge              local
1eb6430c10a2        host                host                local
9297a58de8a4        none                null                local
(base) shushiyada@Khushis-MacBook-Air-2 flask-app % docker network create my-network
45070714afc966b8c3aac31eb6c5eb8edb7eb549d4ebd29a2c3cfe411b4dedd
(base) shushiyada@Khushis-MacBook-Air-2 flask-app % docker network inspect my-network
[
  {
    "Name": "my-network",
    "Id": "45070714afc966b8c3aac31eb6c5eb8edb7eb549d4ebd29a2c3cfe411b4dedd",
    "Created": "2026-03-14T18:16:40.143954879Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.19.0.0/16",
          "IPRange": "",
          "Gateway": "172.19.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {},
    "Containers": {},
    "Status": {
      "IPAM": {
        "Subnets": [
          {
            "172.19.0.0/16": {
              "IPsInUse": 3,
              "DynamicIPsAvailable": 65533
            }
          ]
        }
      }
    }
  }
]
22.20 1
(base) shushiyada@Khushis-MacBook-Air-2 flask-app % docker run -d --name host-app --network host nginx
080403295268f16336d147a2e5d21868c4791576edc72c083379e87301829bf
(base) shushiyada@Khushis-MacBook-Air-2 flask-app % docker run -d --name isolated-app --network none alpine sleep 3600
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
d8ad8cd72000: Pull complete
37093460b0e0: Download complete
c094f30e6ea0: Download complete
Digest: sha256:25189184c71ddad732c0112a8623239086e0a2871e0825f20ecb8f2190c3f659
Status: Downloaded newer image for alpine:latest
094a583305201809c1151ebac9c160808604542a7634c6aba97ac887fc1712
(base) shushiyada@Khushis-MacBook-Air-2 flask-app % docker exec isolated-app ifconfig
lo
  Link encap:Local Loopback
  inet addr:127.0.0.1 Mask:255.0.0.0
  inet6 addr: ::1/128 Scope:Host
  UP LOOPBACK RUNNING MTU:65536 Metric:1
  RX packets:0 errors:0 dropped:0 overruns:0 frame:0
  TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
  collisions:0 txqueuelen:1000
  RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

(base) shushiyada@Khushis-MacBook-Air-2 flask-app % docker exec net-web1 nslookup net-web2
OCI runtime exec failed: exec failed: unable to start container process: exec: "nslookup": executable file not found in $PATH
22.21

```



```
(base) thashiydev@Thushis-MacBook-Air:~$ flask-app % docker exec net-web1 nslookup net-web2
OCI runtime exec failed: exec failed: unable to start container process: exec: "nslookup": executable file not found in $PATH
(base) thashiydev@Thushis-MacBook-Air:~$ flask-app % docker exec net-web1 ping -c 3 net-web2
OCI runtime exec failed: exec failed: unable to start container process: exec: "ping": executable file not found in $PATH
(base) thashiydev@Thushis-MacBook-Air:~$ flask-app % docker network inspect my-network
{
  "Name": "my-network",
  "Id": "45878714afc36608c3a0c31c66c5eb8ed7eb543d4ebd29a2c3cfe411b4de0d",
  "Created": "2026-03-14T18:16:40.143954879Z",
  "Scope": "local",
  "Driver": "bridge",
  "EnableIPv4": true,
  "EnableIPv6": false,
  "IPAM": {
    "Driver": "default",
    "Options": {},
    "Config": [
      {
        "Subnet": "172.19.0.0/16",
        "IPRange": "",
        "Gateway": "172.19.0.1"
      }
    ]
  },
  "Internal": false,
  "Attachable": false,
  "Ingress": false,
  "ConfigFrom": {
    "Network": ""
  },
  "ConfigOnly": false,
  "Options": {
    "com.docker.network.enable_ipv4": "true",
    "com.docker.network.enable_ipv6": "false"
  },
  "Labels": {},
  "Containers": {
    "623bccc36a218a5936f6b0c931b0b6fc649230991830678304d9ba2c477755828": {
      "Name": "net-web2",
      "EndpointID": "bd297c18a3c23e8469dd8174e6ca8cec24a899cd5e7ec847da2f27f6b663a9c6",
      "MacAddress": "ea:22:4d:56:2f:92",
      "IPv4Address": "172.19.0.3/16",
      "IPv6Address": ""
    },
    "63c10432ca9727f92ef4e781443536dbf65d436408905cedcd14127883e573ed8": {
      "Name": "net-web1",
      "EndpointID": "dce449a9e0cb3afbd431b2b7da769991ab711647ad34cf332945763c2f14086d",
      "MacAddress": "c2:d2:88:ac:5d:d1",
      "IPv4Address": "172.19.0.2/16",
      "IPv6Address": ""
    }
  },
  "Status": {
    "IPAM": {
      "Subnets": [
        {
          "172.19.0.0/16": {
            "IPsInUse": 5,
            "DynamicIPsAvailable": 65531
          }
        }
      ]
    }
  }
}
```

22.22		61
22.23	Part 5: Real-World Example	61
22.23.1	Architecture	61
22.23.2	Step 19 — Deploy the Stack	62
22.24		62
22.24.1	Step 20 — Connect Flask and Verify Network	62
22.24.2	Step 20b — Verify Logs	64
22.24.3	Step 21 — Monitor All Services	64
22.25	Cleanup	64
22.26	Errors Encountered & Fixes	66
22.27	Key Takeaways	67
23	Experiment 6 – Docker and Compose Exercises	67
23.1	Aim	67
24	Part 1 – Single Container Comparison	67

24.1	Step 1: Run nginx with docker run	67
24.2	lab-nginx (base) khushivadava@Khushis-MacBook-Air-2 exp-6 % docker stop lab-nginx (base) khushivadava@Khushis-MacBook-Air-2 exp-6 % docker rm lab-nginx lab-nginx	68
24.3	Step 2: Run nginx with docker-compose	68
24.4	Architecture	81
24.5	Process Done	81
24.5.1	Step 1: Jenkins Setup	81
24.5.2	Step 2: Installed Jenkins Plugins	82
24.5.3	Step 3: Added Docker Hub Credentials	82
24.5.4	Step 4: Created GitHub Repository	82
24.5.5	Step 5: Created Pipeline Job	82
24.5.6	Step 6: Configured GitHub Webhook	82
24.6	Jenkinsfile Pipeline	82
24.7	How It Works	83
24.8	Testing	83
24.8.1	Manual Build	83
24.8.2	Automated Trigger	83
24.9	Quick Commands	84
24.10	Key Features	84
24.11	Conclusion	84
24.12	Experiment Steps	87
24.12.1	Step 1: Build the Docker Image from Dockerfile	87
24.12.2	Step 2: Run the Docker Container	88
24.12.3	Step 3: Verify Container Status	88
24.12.4	Step 4: Test SSH Connection to Container	88
24.12.5	Step 5: Configure Ansible Inventory	88
24.12.6	Step 6: Run Ansible Playbook on Container	89
24.12.7	Step 7: Verify Automation Results	89
24.13	Cleanup Commands	89
24.14	Key Learnings	90
24.15	Troubleshooting	90
24.15.1	Issue: Permission Denied (publickey)	90
24.15.2	Issue: Connection Refused	90
24.15.3	Issue: Ansible Connection Failed	90
24.16	Conclusion	90
25	Lab 10: CI/CD Pipeline with Jenkins and SonarQube	90
25.1	Introduction	90
25.2	Prerequisites	90
25.3	Project Structure	90
25.4	Step-by-Step Instructions	91
25.5	Jenkins Pipeline Overview	91
25.6	SonarQube Integration	91
25.7	Screenshots	91
25.8	Troubleshooting	94
25.9	Conclusion	95
26	Lab 11: Deploying WordPress with Docker Swarm	95
26.1	Introduction	95
26.2	Objectives	95
26.3	Prerequisites	95
26.4	Steps	95

26.4.1	1. Initialize Docker Swarm	95
26.4.2	2. Deploy the Stack	95
26.4.3	3. Check Service Status	95
26.4.4	4. Access WordPress	96
26.4.5	5. Scaling Services (Optional)	96
26.5	docker-compose.yml Overview	96
26.6	Troubleshooting	96
26.7	Conclusion	96
26.8	Screenshots	98
27	Kubernetes Lab Experiment – Deploying WordPress using kubeadm	99
27.1	Overview	99
27.2	Objectives	99
27.3	Prerequisites	99
27.4	Cluster Setup using kubeadm	100
27.4.1	Step 1: Initialize the Kubernetes Cluster	100
27.4.2	Step 2: Configure kubectl for the user	100
27.4.3	Step 3: Install Network Plugin (Calico)	100
27.5	Application Deployment (WordPress)	100
27.5.1	Step 4: Deploy WordPress	100
27.5.2	Step 5: Expose WordPress Service	100
27.6	Verification Commands	100
27.6.1	Check Nodes	100
27.6.2	Check Pods	100
27.6.3	Check Services	100
27.7	Scaling the Application	101
27.8	Self-Healing Test	101
27.9	Observations	101
27.10	Issues Faced During Experiment	101
27.11	Conclusion	101

1 Experiment 1

1.0.1 Note:- As I am a mac-user I have performed the experiment on Someone else WINDOWS system.

1.1 Comparison of VMs and containers using Ubuntu and Nginx

1.2 Objective

1. To understand the conceptual and practical differences between Virtual Machine and Container.
2. To install and configure a Virtual Machine using VirtualBox and Vagrant on windows.
3. To install and configure Containers using Docker inside WSL.
4. To deploy on Ubuntu-based Nginx web server in both environments.
5. To compare resource utilization, performance, and operational characteristics of VMs and Containers.

1.3 Theory

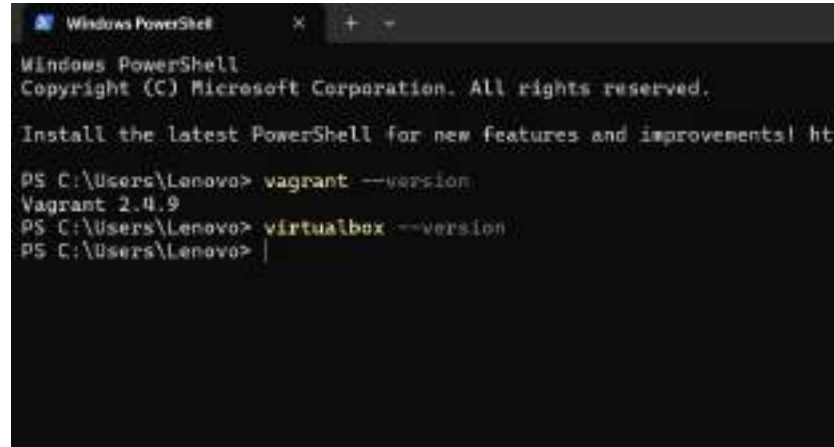
Virtualization and containerization are modern technologies used to run applications in isolated environments. A Virtual Machine (VM) emulates a complete physical system and runs its own operating system using a hypervisor. In this experiment, Ubuntu is installed as a VM using VirtualBox and Vagrant. VMs provide strong isolation but consume more system resources and have slower startup times due to the presence of a full guest OS.

Containers, on the other hand, are lightweight and share the host operating system's kernel. Using Docker

inside WSL, an Ubuntu-based Nginx container is deployed. Containers start faster, use fewer resources, and are highly portable, making them suitable for modern cloud and microservices-based applications.

Ubuntu is used as the base operating system in both environments for consistency, while Nginx is deployed as the web server due to its high performance and low resource usage. By deploying Nginx on both a VM and a container, this experiment compares resource utilization, performance, and operational characteristics of virtual machines and containers.

1.3.1 Step 1: Install VirtualBox and vagrant.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

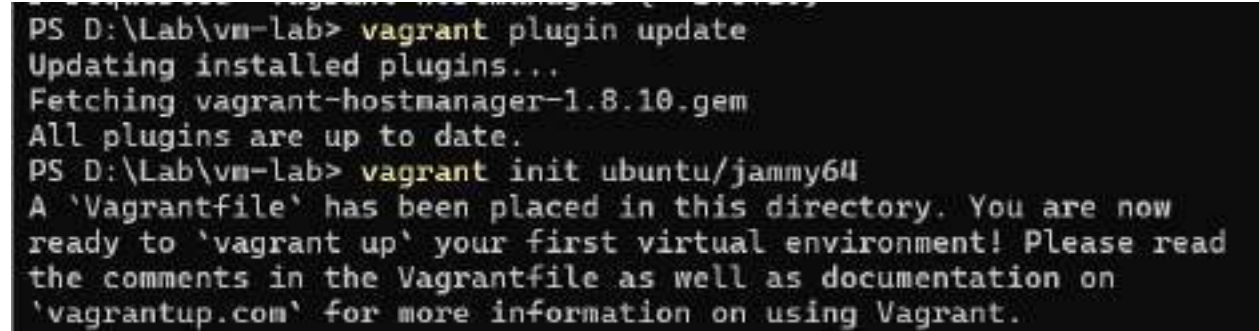
Install the latest PowerShell for new features and improvements! https://aka.ms/PowerShellLatest

PS C:\Users\Lenovo> vagrant --version
Vagrant 2.4.9
PS C:\Users\Lenovo> virtualbox --version
PS C:\Users\Lenovo> |
```

Download and install Oracle VirtualBox and vagrant.

1.3.2 Step 2: Install ubuntu

Install ubuntu



```
PS D:\Lab\vm-lab> vagrant plugin update
Updating installed plugins...
Fetching vagrant-hostmanager-1.8.10.gem
All plugins are up to date.
PS D:\Lab\vm-lab> vagrant init ubuntu/jammy64
A 'Vagrantfile' has been placed in this directory. You are now
ready to 'vagrant up' your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.
```

1.3.3 Step 3: start the VM

Run `vagrant up` to create and boot the Ubuntu virtual machine.

```
PS D:\Lab\vm-lab> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Importing base box 'ubuntu/jammy64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'ubuntu/jammy64' version '20241002.0.0' is up to date...
==> default: Setting the name of the VM: vm-lab_default_1770392497176_25685
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
    default: Adapter 1: nat
==> default: Forwarding ports...
    default: 22 (guest) => 2222 (host) (adapter 1)
==> default: Running 'pre-boot' VM customizations...
==> default: Booting VM...
==> default: Waiting for machine to boot. This may take a few minutes...
    default: SSH address: 127.0.0.1:2222
    default: SSH username: vagrant
    default: SSH auth method: private key
    default: Warning: Connection reset. Retrying...
    default:
    default: Vagrant insecure key detected. Vagrant will automatically replace
ce
    default: this with a newly generated keypair for better security.
    default:
    default: Inserting generated public key within guest...
    default: Removing insecure key from the guest if it's present...
    default: Key inserted! Disconnecting and reconnecting using new SSH key.
...
==> default: Machine booted and ready!
==> default: Checking for guest additions in VM...
    default: The guest additions on this VM do not match the installed versi
on of
    default: VirtualBox! In most cases this is fine, but in rare cases it ca
n
    default: prevent things such as shared folders from working properly. If
```

1.3.4 Step 4: Access the VM

```
PS D:\Lab\vm-lab> vagrant ssh
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-142

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Fri Feb  6 15:43:49 UTC 2

System load:            0.2
Usage of /:             3.9% of 38.70GB
Memory usage:          22%
Swap usage:            0%
Processes:             106
Users logged in:       0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd00::cf:77ff:fe5b:6d0

Expanded Security Maintenance for Applications is n

0 updates can be applied immediately.

Enable ESM Apps to receive additional future securi
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week o
To check for new updates run: sudo apt update
New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.
```

Use `vagrant ssh` to log into the Ubuntu VM.

1.3.5 Step 5: Verifying.

1.3.6 Step 6: Destroying and halting VMs

Use `vagrant halt` and `vagrant destroy` to stop and destroy the VM.

```
PS D:\Lab\vm-lab> vagrant halt
==> default: Attempting graceful shutdown of VM...
PS D:\Lab\vm-lab> vagrant destroy
    default: Are you sure you want to destroy the 'default' VM? [y/N] y
==> default: Destroying VM and associated drives...
PS D:\Lab\vm-lab> |
```

1.3.7 Step 7: Checking wsl

```

vagrant@ubuntu-jammy:~$ sudo systemctl start nginx
vagrant@ubuntu-jammy:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>

```

Figure 1: Step 5

```

PS D:\Lab\vm-lab> wsl --version
WSL version: 2.6.3.0
Kernel version: 6.6.87.2-1
WSLg version: 1.0.71
MSRDC version: 1.2.6353
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.26200.7705

```

Figure 2: Step 7

1.3.8 Step 8: Configuring

Install and configure WSL to run Linux environment on Windows.

```
priyambad@PRIYAMBAD:/mnt/c/Users/Lenovo$ docker --version

The command 'docker' could not be found in this WSL 2 distro.
We recommend to activate the WSL integration in Docker Desktop settings.

For details about using Docker Desktop with WSL 2, visit:

https://docs.docker.com/go/wsl2/

priyambad@PRIYAMBAD:/mnt/c/Users/Lenovo$ sudo apt update
[sudo] password for priyambad:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages 1431 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [138 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en 232 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Component [21.5 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Meta-data [9888 B]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [929 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [19.9 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2411 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [553 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Co
```

1.3.9 Step 9: Install Docker inside WSL

Install Docker Engine inside the WSL Ubuntu environment.

```
priyambad@PRIYAMBAD:/mnt/c/Users/Lenovo$ sudo apt install -y docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base iptables libip4tc2
  libip6tc2 libnetfilter-conntrack3 libnfnetlink0 libnftables1 libnftnl11
  nftables pigz runc ubuntu-fan vim-runtime
Suggested packages:
  ifupdown aufs-tools btrfs-progs cgroupfs-mount | cgroup-lite debootstrap
  docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils
  firewallld
The following NEW packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base docker.io iptables
  libip4tc2 libip6tc2 libnetfilter-conntrack3 libnfnetlink0 libnftables1
  libnftnl11 nftables pigz runc ubuntu-fan
The following packages will be upgraded:
  vim-runtime
1 upgraded, 16 newly installed, 0 to remove and 101 not upgraded.
2 not fully installed or removed.
Need to get 77.1 MB/84.3 MB of archives.
after this operation, 333 MB of additional disk space will be used.
```

1.3.10 Step 10: Verify Docker Installation

Check Docker version and status to confirm successful installation.

```
priyambad@PRIYAMBAD:/mnt/c/Users/Lenovo$ sudo systemctl start docker
priyambad@PRIYAMBAD:/mnt/c/Users/Lenovo$ sudo usermod -aG docker $USER
priyambad@PRIYAMBAD:/mnt/c/Users/Lenovo$ |
```

1.3.11 Step 11: Install Nginx in Container

Install and run Nginx inside the Docker container.

```
priyambad@PRIYAMBAD:/mnt/c/Users/Lenovo$ docker run -d -p 8080:80 --name nginx-container nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
0c8d55a45c8d: Pull complete
46bf3a128c8e: Pull complete
4f4efe82d542: Pull complete
7b6cb8ccac7b: Pull complete
f73486a233fd: Pull complete
47cd486a84ef: Pull complete
bae5a1799a80: Pull complete
Digest: sha256:341bf8f3ce6c5277d6882cf6e1fb0319fa4252add24ab6a8e262e8856d313288
Status: Downloaded newer image for nginx:latest
5772f8e274efc86bf3a85fb789fbc4645a1b7978335b7228c2aab5861831ee99
prijambad@PRIYAMBAD:/mnt/c/Users/Lenovo$ curl localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color:scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

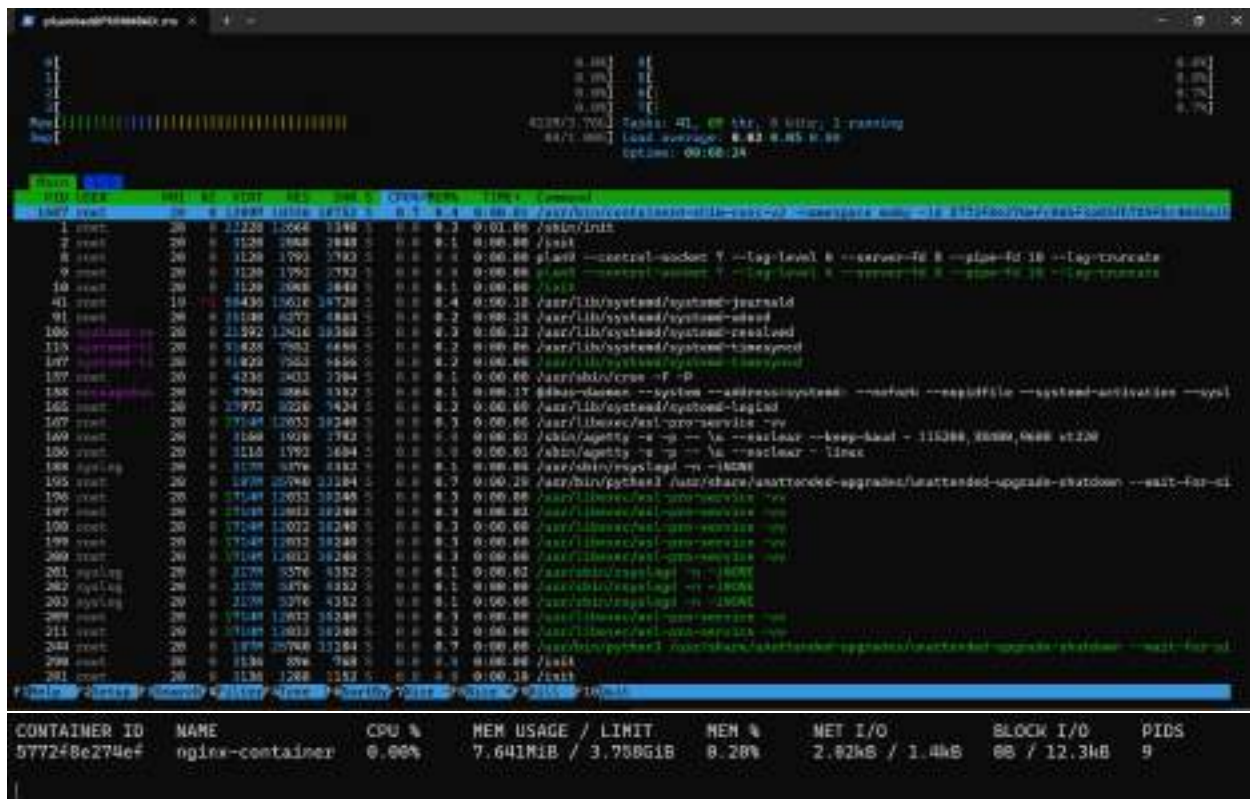
<p><em>Thank you for using nginx.</em></p>
</body>
</html>
prijambad@PRIYAMBAD:/mnt/c/Users/Lenovo$
```

1.3.12 Step 12: Compare VM and Container Performance

Observe startup time, memory usage, CPU consumption, and overall performance.

```
prijambad@PRIYAMBAD:/mnt/c/Users/Lenovo$ free -h
              total        used        free      shared  buff/cache   available
Mem:           3.8Gi        568Mi        1.9Gi        3.6Mi        1.5Gi        3.2Gi
Swap:          1.0Gi           0B        1.0Gi

prijambad@PRIYAMBAD:/mnt/c/Users/Lenovo$ httpd
prijambad@PRIYAMBAD:/mnt/c/Users/Lenovo$ systemd-analyze
Startup finished in 2.202s (userspace)
graphical.target reached after 2.143s in userspace.
prijambad@PRIYAMBAD:/mnt/c/Users/Lenovo$
```



1.4 Result

Ubuntu-based Nginx web server was successfully deployed on both Virtual Machine and Docker Container. The experiment demonstrated that containers are more lightweight and faster, while virtual machines provide stronger isolation with higher resource usage.

1.5 Conclusion

This experiment highlights the key differences between Virtual Machines and Containers in terms of architecture, performance, and resource utilization, helping understand their appropriate use cases in modern computing environments.

2 Experiment 2: Docker Installation, Configuration, and Running Images

2.1 Objective

The objective of this experiment is to:

- Verify Docker installation and configuration on macOS
- Pull a Docker image from Docker Hub
- Run a Docker container with port mapping
- Access the running container using localhost
- Manage the container lifecycle (stop, remove)
- Remove Docker images after use

2.2 Commands Executed

2.2.1 1. Docker Installation & Configuration Check

The following commands were used to verify Docker installation and ensure that the Docker Engine was running:

```
docker --version  
docker info
```

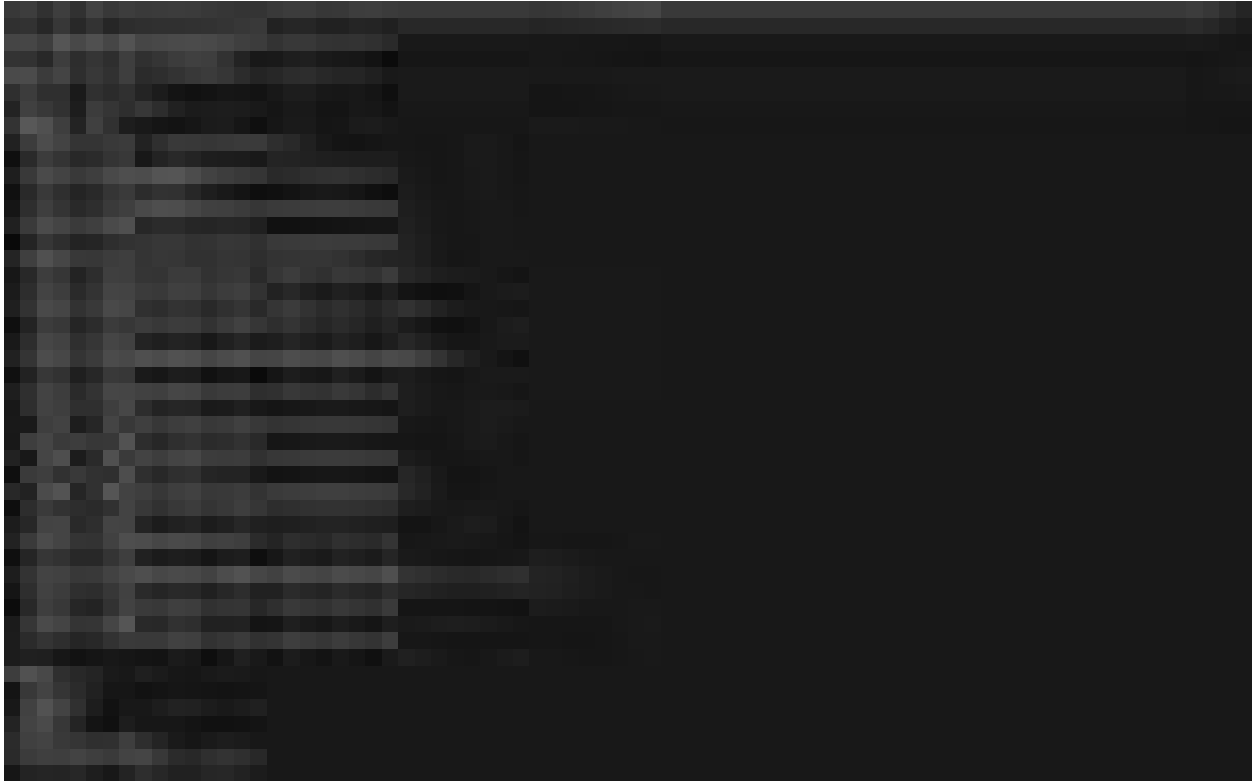


Figure 3: screenshot 1

2.2.2 2. Pull Docker Image

The nginx image was pulled from Docker Hub using the following command:

```
docker pull nginx
```

To verify the downloaded image:

```
docker images
```

2.2.3 3. Run Docker Container with Port Mapping

The nginx container was started in detached mode with port mapping:

```
docker run -d -p 8080:80 nginx
```

To verify the running container:

```
docker ps
```

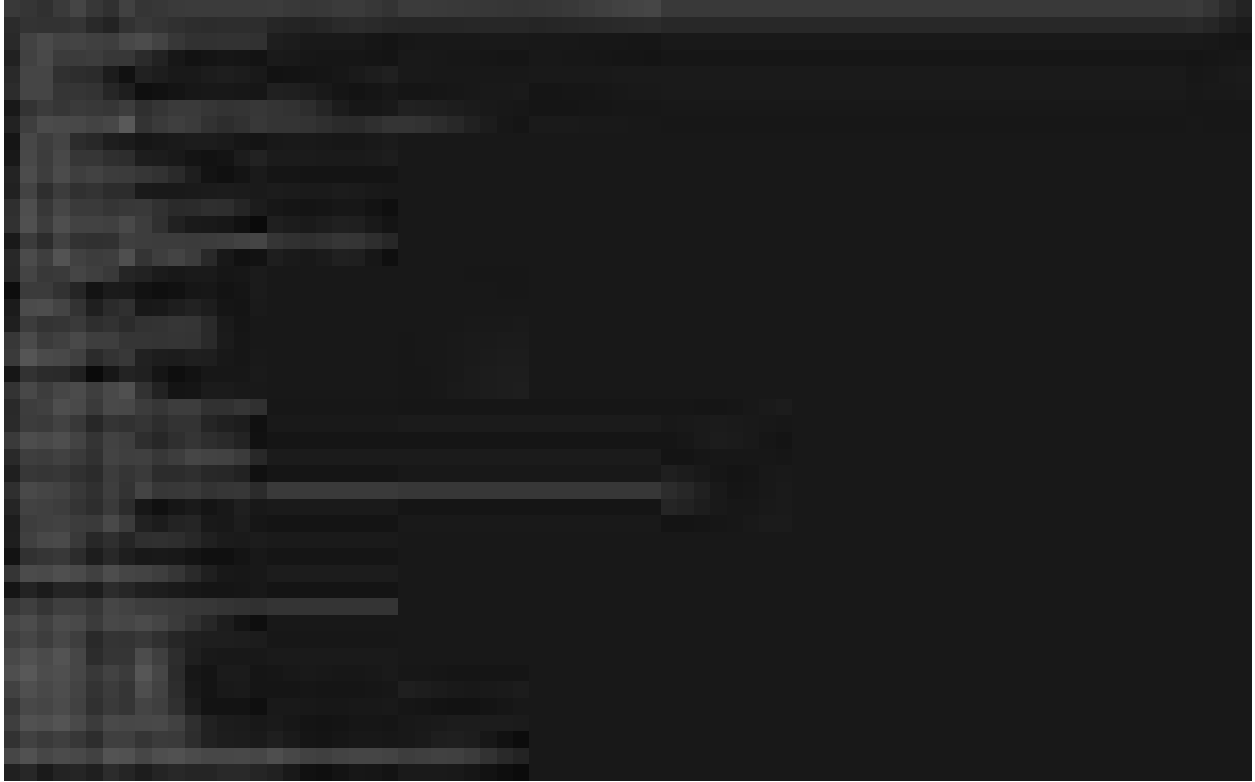



Figure 4: screenshot 2

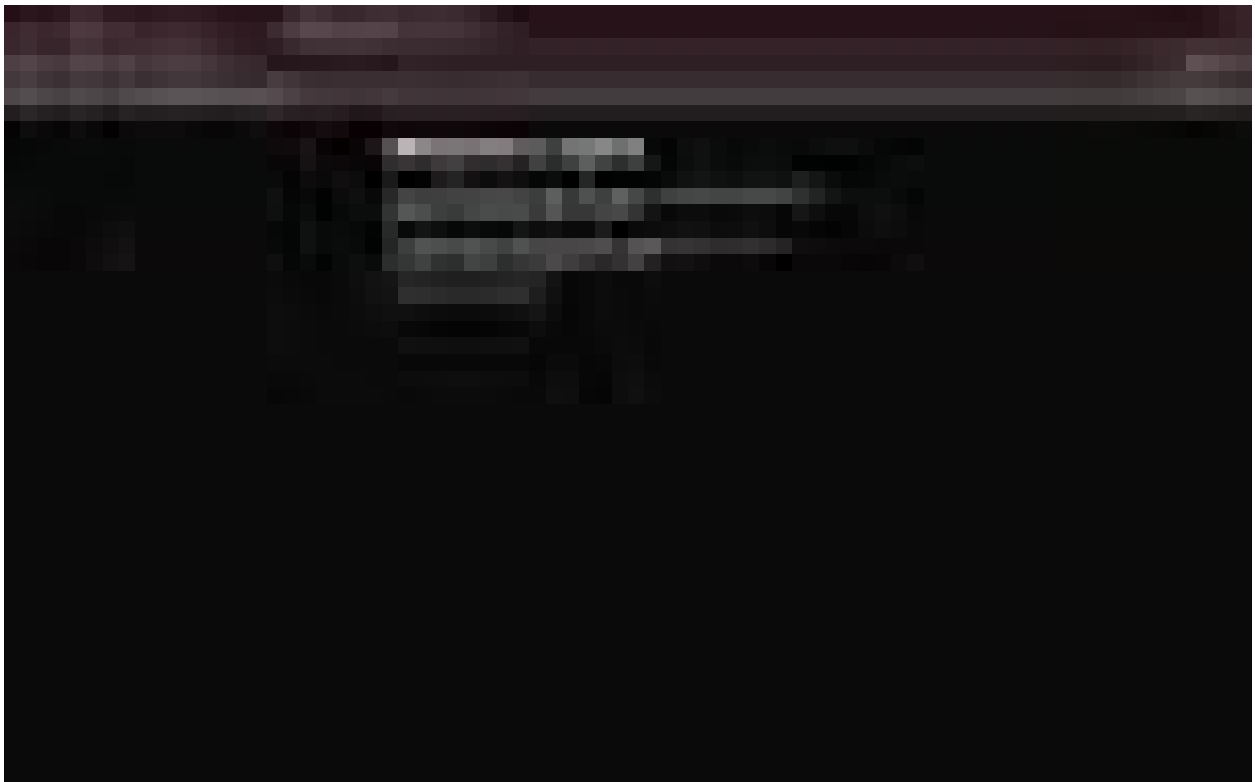


Figure 5: screenshot 3

2.2.4 4. Container Lifecycle Management

The running container was stopped and removed using the following commands:

```
docker stop <container_id>
docker rm <container_id>
```

To verify stopped containers:

```
docker ps -a
```

2.2.5 5. Remove Docker Image

After removing the container, the nginx image was deleted:

```
docker rmi nginx
```

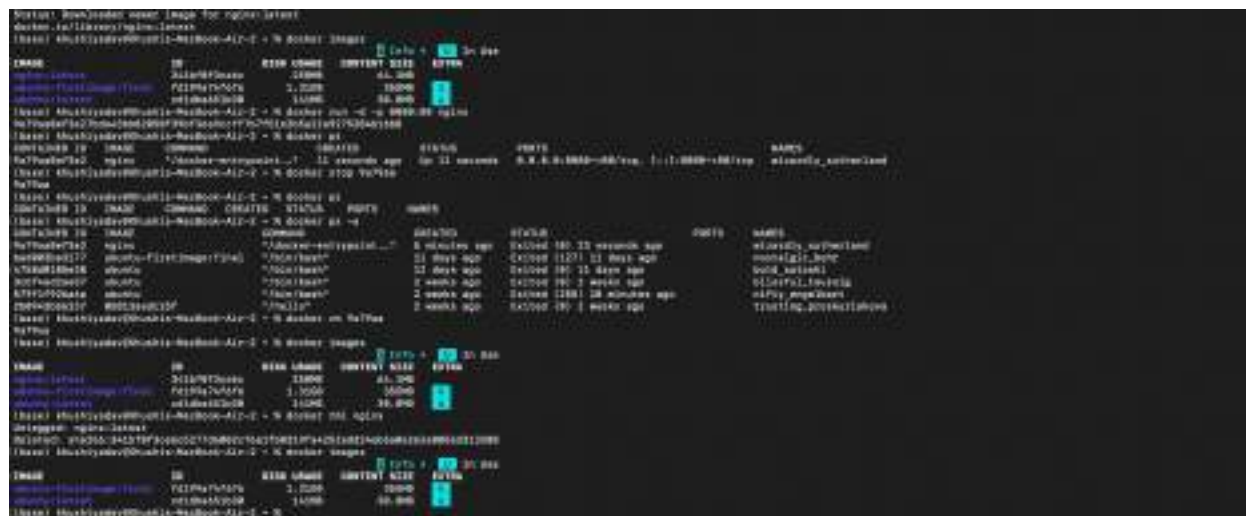


Figure 6: screenshot 4

2.2.6 Result

Docker images were successfully pulled, containers were executed with port mapping, and container lifecycle commands were performed successfully. ### Conclusion This experiment demonstrated the use of Docker for containerization. It showed how Docker images are pulled from Docker Hub, containers are created and accessed using port mapping, and how Docker efficiently manages container lifecycle operations.

3 Experiment 3: Deploying NGINX Using Different Base Images

3.1 Objective

To deploy NGINX using: - Official NGINX Image - Ubuntu-based Custom Image - Alpine-based Custom Image

And compare: - Image size - Configuration behavior - Docker layers - Real-world use cases

4 Prerequisites

- Docker installed and running

- Basic Linux command knowledge
 - Understanding of Dockerfile and port mapping
-

5 Part 1: Deploy NGINX Using Official Image

5.1 Step 1: Pull Image

```
docker pull nginx:latest
```

5.2 Step 2: Run Container

```
docker run -d --name nginx-official -p 8080:80 nginx
```



Figure 7: screenshot 1

5.3 Step 3: Verify

Open in browser:

<http://localhost:8080>

Output:

Welcome to nginx!

5.4



6 Part 2: Custom NGINX Using Ubuntu Base Image

6.1 Step 1: Create Dockerfile

```
FROM ubuntu:22.04
```

```
RUN apt-get update && \  
    apt-get install -y nginx && \  
    apt-get clean && \  
    rm -rf /var/lib/apt/lists/*
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

6.2 Step 2: Build Image

```
docker build -t nginx-ubuntu .
```

6.3 Step 3: Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

Verify in browser:

<http://localhost:8081>

Output:

Welcome to nginx!

7.2 Step 2: Build Image

```
docker build -t nginx-alpine .
```

7.3 Step 3: Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

[illegible]

7.4

8 Issue Faced in Alpine

After running:

http://localhost:8082

It showed:

404 Not Found



8.1

8.2 Debugging Process

8.2.1 Checked Container

```
docker ps
```

Container was running.

8.2.2 Entered Container

```
docker exec -it nginx-alpine sh
```

8.2.3 Checked Default Config

```
cat /etc/nginx/http.d/default.conf
```

Found:

```
server {  
    listen 80 default_server;  
  
    location / {  
        return 404;  
    }  
}
```

Alpine default configuration intentionally returns 404.

9 Fix Applied

9.0.1 Created Web Directory

```
mkdir -p /var/www/html
echo "<h1>Hello from Alpine NGINX</h1>" > /var/www/html/index.html
```

9.0.2 Modified Configuration

```
server {
    listen 80 default_server;

    root /var/www/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

9.0.3 Tested Configuration

```
nginx -t
```

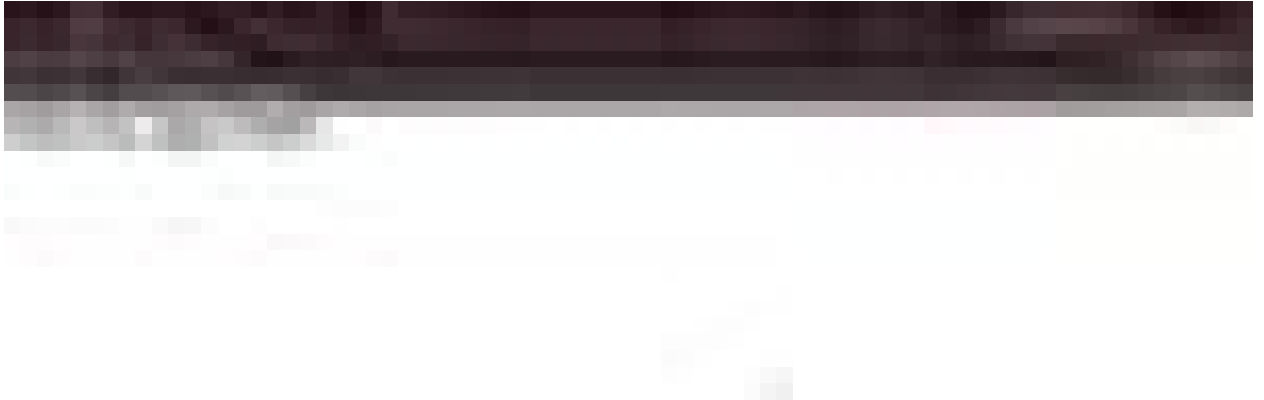
Output:

```
syntax is ok
test is successful
```

9.0.4 Reloaded NGINX

```
nginx -s reload
```


Hello from Alpine NGINX



9.1

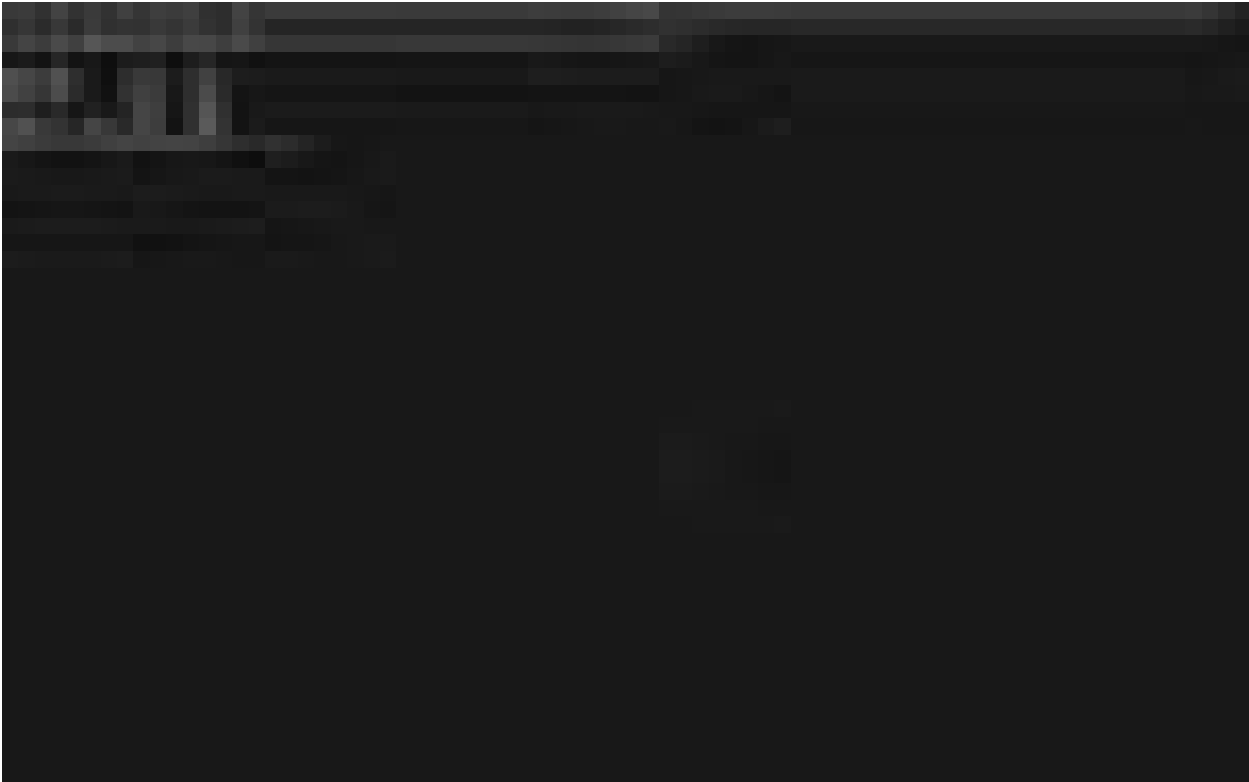
10 Image Size Comparison

Command used:

```
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
```

10.1 Results:

Image Type	Size
nginx	258MB
nginx-ubuntu	187MB
nginx-alpine	16.7MB



10.2

11 Observations

- Alpine image is the smallest (16.7MB)
 - Ubuntu image is larger due to full OS dependencies
 - Official image is optimized but heavier than Alpine
 - Alpine default configuration returns 404 intentionally
 - Base image selection impacts:
 - Security
 - Performance
 - Image size
 - Startup time
-

12 Conclusion

In this experiment, NGINX was deployed using three different base images. The experiment demonstrated that base image selection significantly impacts container size and configuration behavior. Alpine-based images are lightweight and ideal for microservices and cloud environments, while Ubuntu-based images provide more debugging tools but increase image size. Official images are production-ready but may include additional modules.

This experiment highlights the importance of optimizing container images for performance and security in real-world deployments.

13 Cleanup Commands

```
docker stop nginx-official nginx-ubuntu nginx-alpine
docker rm nginx-official nginx-ubuntu nginx-alpine
docker rmi nginx nginx-ubuntu nginx-alpine
docker system prune -a
```

14 Key Learnings

- Docker image layering
- Base image impact on size
- NGINX configuration basics
- Debugging containerized applications
- Production vs minimal container strategies

15 Experiment 4 – Docker Essentials

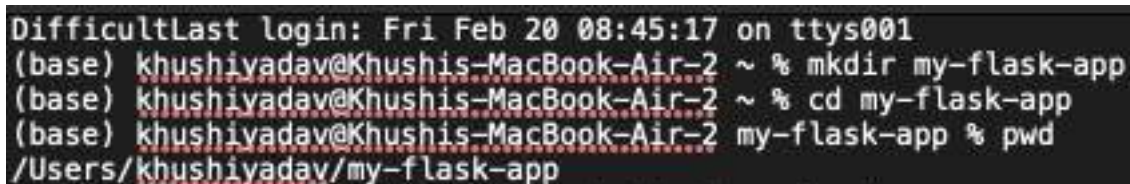
15.1 Aim

To containerize a Flask application using Docker, manage containers, and push the image to Docker Hub.

16 Part 1 – Containerizing a Flask Application

16.1 Step 1: Create Project Directory

```
mkdir my-flask-app
cd my-flask-app
```



The screenshot shows a terminal window with the following text: "DifficultLast login: Fri Feb 20 08:45:17 on ttys001", "(base) khushiyaadav@Khushis-MacBook-Air-2 ~ % mkdir my-flask-app", "(base) khushiyaadav@Khushis-MacBook-Air-2 ~ % cd my-flask-app", "(base) khushiyaadav@Khushis-MacBook-Air-2 my-flask-app % pwd", and "/Users/khushiyaadav/my-flask-app".

Figure 9: screenshot 1

16.2 Step 2: Create Flask Application (app.py)

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
```

```
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

16.3 Step 3: Create requirements.txt

Flask==2.3.3

16.4 Step 4: Create Dockerfile

```
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```



The screenshot shows a terminal window with the following commands and outputs:

```
(base) khushiavadav@Khushis-MacBook-Air-2 my-flask-app % nano app.py
(base) khushiavadav@Khushis-MacBook-Air-2 my-flask-app % ls
app.py
(base) khushiavadav@Khushis-MacBook-Air-2 my-flask-app % nano requirements.txt
(base) khushiavadav@Khushis-MacBook-Air-2 my-flask-app % ls
app.py requirements.txt
(base) khushiavadav@Khushis-MacBook-Air-2 my-flask-app % nano Dockerfile
(base) khushiavadav@Khushis-MacBook-Air-2 my-flask-app % nano Dockerfile
(base) khushiavadav@Khushis-MacBook-Air-2 my-flask-app % ls
Dockerfile app.py requirements.txt
```

Figure 10: screenshot 1

16.5 Step 5: Build Docker Image

```
docker build -t my-flask-app .
```

16.6 Step 6: Run Container

```
docker run -d -p 5001:5000 --name flask-container my-flask-app
```

Access application:

<http://localhost:5001>

```

(base) khushiyadav@Khushis-MacBook-Air:~$ docker build -t my-flask-app .
ERROR: Cannot connect to the Docker daemon at unix:///Users/khushiyadav/.docker/run/docker.sock. Is the docker daemon running?
(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % docker build -t my-flask-app .
[+] Building 75.1s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 361B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> transferring context: 20
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2097f6918b16bd13 62.2s
=> resolve docker.io/library/python:3.9-slim@sha256:2097f6918b16bd13
=> sha256:c21f463847388e81a1da5cd0266dcf8e46671802a22 251B / 251B 0.7s
=> sha256:e0fe7d4fab581a61a5ba6d1e1c2956e0a05d3c 13.83MB / 13.83MB 38.7s
=> sha256:47968ad8cc3edbea82d4959dfbdc65226d5c55ef 1.27MB / 1.27MB 38.7s
=> sha256:a18e551102678581ec8359c78ab9eebfbf2af546 38.14MB / 38.14MB 61.4s
=> extracting sha256:a18e551102678581ec8359c78ab9eebfbf2af546ff7c25b3 0.5s
=> extracting sha256:47968ad8cc3edbea82d4959dfbdc65226d5c55ef3f3691 0.0s
=> extracting sha256:e0fe7d4fab581a61a5ba6d1e1c2956e0a05d3c382718a 0.2s
=> extracting sha256:c21f463847388e81a1da5cd0266dcf8e46671802a22 0.0s
=> [internal] load build context
=> transferring context: 321B
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY app.py .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:8e7de8cdf73969cf174a82b6e64fe7e906e18bb 0.0s
=> exporting config sha256:843f408fc854be25130613e4d77b772bdc0c27110 0.0s
=> exporting attestation manifest sha256:6d289c33f3c26386e08553a4867 0.0s
=> exporting manifest list sha256:7cbfa25618ecb0a2777fe717a4b32c01b3 0.0s
=> naming to docker.io/library/my-flask-app:latest
=> unpacking to docker.io/library/my-flask-app:latest 0.1s

```

Figure 11: screenshot 1

```

(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % docker run -d -p 5000:5000 --name flask-container my-flask-app
d163d9c99ad6b588e73de5f8c0a252118cc836460db23204b728ba2e3278981
docker: Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:5000 -> 127.0.0.1:0: Listen tcp 0.0.0.0:5000: bind:
Run 'docker run --help' for more information
(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % cd ..
(base) khushiyadav@Khushis-MacBook-Air:~$ % docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
(base) khushiyadav@Khushis-MacBook-Air:~$ % lsuf -i 15000
COMMAND PID      USER      FD      TYPE          DEVICE SIZE/OFF NODE NAME
ControlC 422 khushiyadav 10u IPv4 8ad1c40b78d2d6e2fe 0B TCP *:connex:main (LISTEN)
ControlC 422 khushiyadav 11u IPv6 8x5e798f2020e6470c 0B TCP *:connex:main (LISTEN)
(base) khushiyadav@Khushis-MacBook-Air:~$ % cd my-flask-app
(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % docker run -d -p 5001:5000 --name flask-container my-flask-app
docker: Error response from daemon: Conflict. The container name "/flask-container" is already in use by container "d163d9c99ad6b588e73de5f8c0a252118cc836460db23204b728ba2e3278981" that container is not able to reuse that name.
Run 'docker run --help' for more information
(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
d163d9c99ad        my-flask-app       "python app.py"     9 minutes ago       Created
826ad8172ac3        ubuntu             "bash"             2 days ago          Exited (255) 13 minutes ago      flask-container
3be9f9811596        ubuntu             "/bin/bash"        2 days ago          Exited (0) 2 days ago            keen_cerf
8c8f7423b838        ubuntu             "/bin/bash"        2 days ago          Exited (0) 2 days ago            leahina_brownmawata
8alac74c8d86        ubuntu             "/bin/bash"        2 days ago          Exited (0) 2 days ago            lucid_lawvitt
88f99b7ede0e        ubuntu             "/bin/bash"        2 days ago          Exited (0) 2 days ago            keen_wozniak
8ed083e3177        ubuntu:latest       "/bin/bash"        3 weeks ago         Exited (127) 3 weeks ago         oostalgic_hunt
4758d0185e34        ubuntu             "/bin/bash"        3 weeks ago         Exited (0) 3 weeks ago            bold_satoshi
3cbf4d20057        ubuntu             "/bin/bash"        3 weeks ago         Exited (0) 3 weeks ago            blissful_tanaka
579f1f32a6a        ubuntu             "/bin/bash"        4 weeks ago         Exited (255) 11 days ago         nifty_casabari
2b0443b6237        85413aedc15f       "/hello"           4 weeks ago         Exited (0) 4 weeks ago          tration_procuriakova

```

Figure 12: screenshot 1



Figure 13: screenshot 1

16.7 Step 7: Test Health Endpoint

`http://localhost:5001/health`

16.8 Step 8: View Logs

`docker logs flask-container`

16.9

```
(base) khushiyadv@Khushis-MacBook-Air:~$ my-flask-app % docker logs flask-container
* Serving Flask app "app"
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
192.168.65.1 - - [20/Feb/2020 03:37:34] "GET / HTTP/1.1" 200 -
192.168.65.1 - - [20/Feb/2020 03:37:34] "GET /favicon.ico HTTP/1.1" 404 -
192.168.65.1 - - [20/Feb/2020 03:39:02] "GET /health HTTP/1.1" 200 -
(base) khushiyadv@Khushis-MacBook-Air:~$ my-flask-app % docker stop flask-container
flask-container
(base) khushiyadv@Khushis-MacBook-Air:~$ my-flask-app % docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
(base) khushiyadv@Khushis-MacBook-Air:~$ my-flask-app % docker start flask-container
flask-container
(base) khushiyadv@Khushis-MacBook-Air:~$ my-flask-app % docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
817fc12ff036       my-flask-app       "python app.py"     4 minutes ago       Up 8 seconds       0.0.0.0:5001->5000/tcp, :::5001->5000/tcp   flask-container
```

16.10 Step 9: Stop, Start and Remove Container

```
docker stop flask-container
docker start flask-container
docker rm -f flask-container
```



Figure 14: screenshot 1

```
(base) khushi@khushi-MacBook-Air-2 my-flask-app % docker stop flask-container
flask-container
(base) khushi@khushi-MacBook-Air-2 my-flask-app % docker rm flask-container
flask-container
(base) khushi@khushi-MacBook-Air-2 my-flask-app % docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
83d3d3872ac8	ubuntu	"/bin/bash"	2 days ago	Exited (255) 21 minutes ago		class
35a8f3611985	ubuntu	"/bin/bash"	2 days ago	Exited (8) 2 days ago		kees_cerf
bc8f7423bd88	ubuntu	"/bin/bash"	2 days ago	Exited (8) 2 days ago		lowahiro_3c0b0a0a0a
8a1a074c0b86	ubuntu	"/bin/bash"	2 days ago	Exited (8) 2 days ago		lucid_image15
86f9932ed08c	ubuntu	"/bin/bash"	2 days ago	Exited (8) 2 days ago		kees_w00t14h
6c00d3bc3177	ubuntu:firstimage:final	"/bin/bash"	3 weeks ago	Exited (127) 3 weeks ago		postg0lc_b00f
475d3d185e38	ubuntu	"/bin/bash"	3 weeks ago	Exited (8) 3 weeks ago		bold_satoshi
3c7f462ae17	ubuntu	"/bin/bash"	3 weeks ago	Exited (8) 3 weeks ago		blissful_tausssg
579f1f926a6a	ubuntu	"/bin/bash"	4 weeks ago	Exited (255) 11 days ago		nitty_engelb0rt
2b6943bb6237	85813aedc15f	"/hello"	4 weeks ago	Exited (8) 4 weeks ago		truxtd0w_4cc0w4r10h0x

```
(base) khushi@khushi-MacBook-Air-2 my-flask-app % docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
my-flask-app:latest	7c6fa35818ec	228MB	49.1MB	
ubuntu:firstimage:final	f2199a7416f0	1.31GB	300MB	0
ubuntu:latest	c81d9a811b38	143MB	30.8MB	0

```
(base) khushi@khushi-MacBook-Air-2 my-flask-app % username/repository:tag
ssh: no such file or directory: username/repository:tag
```

16.11

17 Part 2 – Docker Hub Operations

17.1 Step 10: Tag Image

```
docker tag my-flask-app ky270405/my-flask-app:v1
```

17.2 Step 11: Login to Docker Hub

```
docker login
```


17.3 Step 12: Push Image

```
docker push ky270405/my-flask-app:v1
```

```
(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % docker tag my-flask-app ky270405/my-flask-app:v1
(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % docker images

IMAGE                ID                                DISK USAGE  CONTENT SIZE  EXTRA
ky270405/my-flask-app:v1  7cbfa35818ec                    220MB       49.2MB
my-flask-app:latest      7cbfa35818ec                    220MB       49.2MB
ubuntu:latest           f6199a746f6                     1.31GB      353MB      U
ubuntu:latest           f6199a746f6                     1.31GB      353MB      U
(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % docker login
Authenticating with existing credentials... [Username: ky270405]

1 Info - To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % docker push ky270405/my-flask-app:v1
The push refers to repository [docker.io/ky270405/my-flask-app]
78ef49702f3c: Pushed
c3f48583473c: Pushed
5c3481acc0d8: Pushed
c541d765add7: Pushed
628b46f4885f: Pushed
926c27388cc7: Pushed
a16e53119287: Pushed
479b4ad0bc1: Pushed
ebfe7d4fa9c5: Pushed
v1 digest: sha256:7cbfa35818ecba7777fe717a4b32c63b385422915485daba3f4dc887b0aac678 size: 858
```

Figure 15: screenshot 1

17.4 Step 13: Remove Local Image

```
docker rmi ky270405/my-flask-app:v1
docker rmi my-flask-app
```

17.5 Step 14: Pull Image from Docker Hub

```
docker pull ky270405/my-flask-app:v1
```

```
(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % docker rmi ky270405/my-flask-app:v1
docker rmi my-flask-app
Untagged: ky270405/my-flask-app:v1
Untagged: my-flask-app:latest
Deleted: sha256:7cbfa35818ecba7777fe717a4b32c63b385422915485daba3f4dc887b0aac678
(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % docker images

IMAGE                ID                                DISK USAGE  CONTENT SIZE  EXTRA
ubuntu:latest           f6199a746f6                     1.31GB      353MB      U
ubuntu:latest           f6199a746f6                     1.31GB      353MB      U
(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % docker pull ky270405/my-flask-app:v1
v1: Pulling from ky270405/my-flask-app
628b46f4885f: Pull complete
926c27388cc7: Pull complete
5c3481acc0d8: Pull complete
78ef49702f3c: Pull complete
c541d765add7: Download complete
Digest: sha256:7cbfa35818ecba7777fe717a4b32c63b385422915485daba3f4dc887b0aac678
Status: Downloaded newer image for ky270405/my-flask-app:v1
docker.io/ky270405/my-flask-app:v1
(base) khushiyadav@Khushis-MacBook-Air:~$ my-flask-app % docker images

IMAGE                ID                                DISK USAGE  CONTENT SIZE  EXTRA
ky270405/my-flask-app:v1  7cbfa35818ec                    220MB       49.2MB
ubuntu:latest           f6199a746f6                     1.31GB      353MB      U
ubuntu:latest           f6199a746f6                     1.31GB      353MB      U
```

Figure 16: screenshot 1

17.6 Step 15: Run Pulled Image

```
docker run -d -p 5002:5000 --name flask-final ky270405/my-flask-app:v1
```



```
(base) khrushiyadev@Khrushis-MacBook-Air-2 my-flask-app % docker run -d -p 5002:5000 --name flask-final ky279485/my-flask-app:v1
c8c09f1650d47a8a5b1f93e15f282e47298c544e0b4de3e73e49878c0df0e77
(base) khrushiyadev@Khrushis-MacBook-Air-2 my-flask-app % docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
c8c09f1650d4        ky279485/my-flask-app:v1   "python app.py"     15 seconds ago      Up 14 seconds      0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp   flask-final
(base) khrushiyadev@Khrushis-MacBook-Air-2 my-flask-app % docker stop flask-final
flask-final
(base) khrushiyadev@Khrushis-MacBook-Air-2 my-flask-app % docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
c8c09f1650d4        ky279485/my-flask-app:v1   "python app.py"     2 minutes ago       Exited (137) 15 seconds ago                  flask-final
824e4d817a43        ubuntu             "bash"             2 days ago          Exited (255) 42 minutes ago                  class
3ba0f8811986        ubuntu            "/bin/bash"        2 days ago          Exited (0) 2 days ago                        keep-cert
bc6f7423b038        ubuntu            "/bin/bash"        2 days ago          Exited (0) 2 days ago                        lauching-bronzequester
8a1ae74c5b86        ubuntu            "/bin/bash"        2 days ago          Exited (0) 2 days ago                        lucie_lervall
55f02b2edc8c        ubuntu            "/bin/bash"        2 days ago          Exited (0) 2 days ago                        nash_mission
bed981be3177        ubuntu-faradaygo:final  "/bin/bash"        3 weeks ago         Exited (127) 3 weeks ago                     nostalgic-horor
475048106e38        ubuntu            "/bin/bash"        3 weeks ago         Exited (0) 3 weeks ago                       bold-satoshi
3cbf4e62ae57        ubuntu            "/bin/bash"        3 weeks ago         Exited (0) 3 weeks ago                       bluesoul-thousand
579f17926a8a        ubuntu            "/bin/bash"        4 weeks ago         Exited (255) 11 days ago                     nifty-magicalact
2b0941b0e237        85813aedc15f        "/hello"           4 weeks ago         Exited (0) 4 weeks ago                       ltravelpz-pronker148000
(base) khrushiyadev@Khrushis-MacBook-Air-2 my-flask-app %
```

Figure 17: screenshot 1

Access:

<http://localhost:5002>

18 Errors and Issues Faced

18.1 -> Docker Daemon Not Running At 9 AM

Error:

Cannot connect to the Docker daemon...

Reason: Docker Desktop was not started.

Solution: Started Docker Desktop and verified using:

docker version

18.2 -> Port 5000 Already in Use

Error:

bind: address already in use

Reason: Port 5000 was already used by a system process.

Solution: Used a different port:

docker run -d -p 5001:5000 ...

18.3 -> Container Name Already in Use

Error:

container name is already in use

Reason: A container with the same name already existed.

Solution:

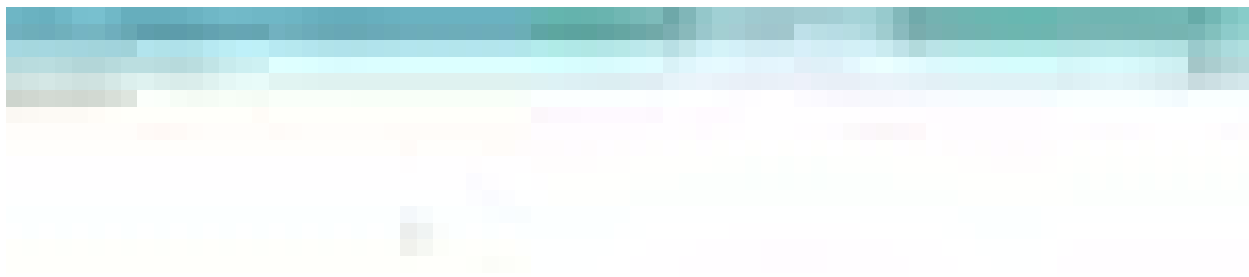


Figure 18: screenshot 6

```
docker rm -f flask-container
```

19 Results

- Successfully containerized a Flask application.
 - Built Docker image.
 - Ran container with port mapping.
 - Managed container lifecycle.
 - Tagged and pushed image to Docker Hub.
 - Pulled and ran image successfully.
-

20 Conclusion

The experiment successfully demonstrated Docker containerization, image management, port mapping, container lifecycle management, and integration with Docker Hub.

21 Experiment 5:

22 Docker — Volumes, Environment Variables, Monitoring & Networks

Platform: MacBook Air M1 | **Date:** March 14–15, 2026

Purpose: Learn essential Docker features for building, configuring, and managing production-ready containerized applications.

22.1 Table of Contents

- Overview
 - Part 1: Docker Volumes
 - Part 2: Environment Variables
 - Part 3: Docker Monitoring
 - Part 4: Docker Networks
 - Part 5: Real-World Example
 - Cleanup
 - Errors Encountered & Fixes
 - Key Takeaways
-

22.2 Overview

Part	Topic	What Was Done
Part 1	Docker Volumes	Anonymous, Named & Bind Mounts with persistence demo
Part 2	Environment Variables	-e flag, -env-file, Flask app configuration
Part 3	Docker Monitoring	stats, logs, top, inspect, system df
Part 4	Docker Networks	Bridge, Host, None & Custom networks
Part 5	Real-World App	Flask + PostgreSQL + Redis multi-container setup

22.3 Part 1: Docker Volumes

Docker containers are ephemeral — all data written inside a container is lost when it is removed. Docker Volumes solve this by persisting data outside the container lifecycle.

22.3.1 Lab 1: Understanding Data Persistence

22.3.1.1 Step 1 — Verify Docker Installation

```
docker --version
docker ps
```

Docker Desktop must be running before executing any docker commands.

22.3.1.2 Step 2 — Create Container and Write Data

```
docker run -it --name test-container ubuntu /bin/bash
```

Inside container:

```
mkdir /data && echo "Hello World" > /data/message.txt
```

```
cat /data/message.txt
```

Output: Hello World

```
exit
```

```
Last login: Fri Mar 13 21:39:38 on console
[base] khushiyadav@Khushis-MacBook-Air-2 ~ % docker ps
Cannot connect to the Docker daemon at unix:///Users/khushiyadav/.docker/run/docker.sock. Is the docker daemon running?
[base] khushiyadav@Khushis-MacBook-Air-2 ~ % docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS                    NAMES
9447b72e771a   wordpress:latest    "docker-entrypoint.s~"   2 weeks ago   Up 12 seconds   80/tcp                  docker-compose-demo-wordpress-1
6ed196e4279e   wordpress:latest    "docker-entrypoint.s~"   2 weeks ago   Up 12 seconds   80/tcp                  docker-compose-demo-wordpress-2
332d11e198d    mysql:5.6            "docker-entrypoint.s~"   2 weeks ago   Up 12 seconds   3306/tcp, 33060/tcp     mysql
[base] khushiyadav@Khushis-MacBook-Air-2 ~ % docker run -it --name test-container ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
66a4bbfab40: Full complete
9c2a2bc78563: Download complete
Digest: sha256:d1e2f52c875c5ca139d51a148fff96f84315c87dce283eah2897c7e95eff4f5
Status: Downloaded newer image for ubuntu:latest
root@71d388b8aa76:/# mkdir /data && echo "Hello World" > /data/message.txt && cat /data/message.txt
Hello World
root@71d388b8aa76:/# exit
exit
```

22.4

22.4.0.1 Step 3 — Prove Data is Ephemeral

Restart and check - data still exists (container just stopped)

```
docker start test-container
```

```
docker exec test-container cat /data/message.txt
```

Now remove and recreate

```
docker stop test-container
```

```
docker rm test-container
```

```
docker run -it --name test-container ubuntu /bin/bash
```

```
cat /data/message.txt
```

ERROR: No such file or directory + data is gone!

```
exit
```

This proves container data does NOT survive removal.

```
[base] khushiyadav@Khushis-MacBook-Air-2 ~ % docker start test-container
test-container
[base] khushiyadav@Khushis-MacBook-Air-2 ~ % docker exec test-container cat /data/message.txt
Hello World
[base] khushiyadav@Khushis-MacBook-Air-2 ~ % docker rm test-container
Error response from daemon: cannot remove container "test-container": container is running: stop the container before removing or force remove
[base] khushiyadav@Khushis-MacBook-Air-2 ~ % docker stop test-container
test-container
[base] khushiyadav@Khushis-MacBook-Air-2 ~ % docker run -it --name test-container ubuntu /bin/bash
root@b0c8ee8b3865:/# cat /data/message.txt
cat: /data/message.txt: No such file or directory
root@b0c8ee8b3865:/# exit
exit
```

22.5

22.5.0.1 Error: Cannot remove running container

Error response from daemon: cannot remove container "test-container":

container is running: stop the container before removing or force remove

22.5.0.2 Fix:

```
docker stop test-container
```

```
docker rm test-container
```

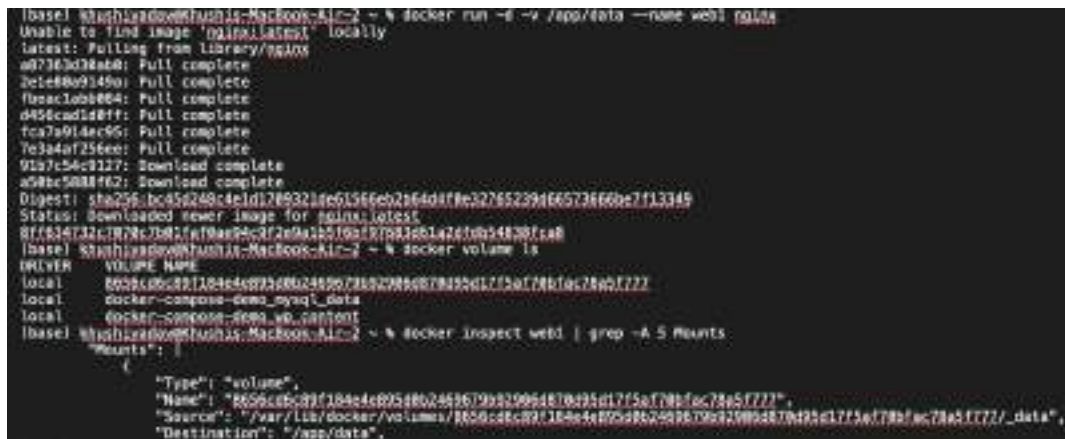
22.5.1 Lab 2: Volume Types

22.5.1.1 Step 4 — Anonymous Volumes

```
docker run -d -v /app/data --name web1 nginx
```

```
docker volume ls
```

```
docker inspect web1 | grep -A 5 Mounts
```



```
(base) khushiyadav@Khushia-MacBook-Air-2 ~ % docker run -d -v /app/data --name web1 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
a07163d38a4d: Pull complete
2efc86a9349a: Pull complete
f9ac10bb864: Pull complete
d456cad188ff: Pull complete
fca7a914ac95: Pull complete
7e3a4af256ee: Pull complete
91a7c54c3227: Download complete
a58bc5088f62: Download complete
Digest: sha256:bc45a248c4e1d17893218e61566eb2b64dffe32765239466573866be7f13349
Status: Downloaded newer image for nginx:latest
8f763473217878c7b81fa79a894c72e0a12519a1577b81c6147c0b54830fca8
(base) khushiyadav@Khushia-MacBook-Air-2 ~ % docker volume ls
DRIVER      VOLUME NAME
local       8056c8c89f184a4e05d862469679b82996487b491d17f5af78bfec78a5f777/_data
local       docker-compose-demo_mysql_data
local       docker-compose-demo_wd_content
(base) khushiyadav@Khushia-MacBook-Air-2 ~ % docker inspect web1 | grep -A 5 Mounts
"Mounts": [
  {
    "Type": "volume",
    "Name": "8056c8c89f184a4e05d862469679b82996487b491d17f5af78bfec78a5f777/_data",
    "Source": "/var/lib/docker/volumes/8056c8c89f184a4e05d862469679b82996487b491d17f5af78bfec78a5f777/_data",
    "Destination": "/app/data",
```

Figure 19: screenshot 4

Output: A random hash name is auto-generated for the anonymous volume, mounted at `/app/data`.

22.5.1.2 Step 5 — Named Volumes

```
docker volume create mydata
```

```
docker run -d -v mydata:/app/data --name web2 nginx
```

```
docker volume ls
```

```
docker volume inspect mydata
```



```
(base) khushiyadav@Khushia-MacBook-Air-2 ~ % docker volume create mydata
mydata
(base) khushiyadav@Khushia-MacBook-Air-2 ~ % docker run -d -v mydata:/app/data --name web2 nginx
8056c8c89f184a4e05d862469679b82996487b491d17f5af78bfec78a5f777
(base) khushiyadav@Khushia-MacBook-Air-2 ~ % docker volume ls
DRIVER      VOLUME NAME
local       8056c8c89f184a4e05d862469679b82996487b491d17f5af78bfec78a5f777/_data
local       docker-compose-demo_mysql_data
local       docker-compose-demo_wd_content
local       mydata
(base) khushiyadav@Khushia-MacBook-Air-2 ~ % docker volume inspect mydata
[
  {
    "CreatedAt": "2025-03-14T17:39:33Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
    "Name": "mydata",
    "Options": null,
    "Scope": "local"
  }
]
```

Output:

```
{
  "Name": "mydata",
  "Driver": "local",
  "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
```

```
"Scope": "local"
}
```

22.5.1.3 Step 6 — Bind Mounts (Host Directory)

```
mkdir ~/myapp-data
docker run -d -v ~/myapp-data:/app/data --name web3 nginx
```

Add file on host

```
echo "From Host" > ~/myapp-data/host-file.txt
```

Verify inside container

```
docker exec web3 cat /app/data/host-file.txt
```

Output: From Host

Bind mounts enable live sync between Mac host and container — any file added on the host is immediately visible inside the container.



```
(base) khushiyadave@Khushis-MacBook-Air-2 ~ % mkdir ~/myapp-data
(base) khushiyadave@Khushis-MacBook-Air-2 ~ % docker run -d -v ~/myapp-data:/app/data --name web3 nginx
8fc7bbe0rd62a774fd2494c8dd817c59271be8471ed2bb93288c5f9195e2952
(base) khushiyadave@Khushis-MacBook-Air-2 ~ % echo "From Host" > ~/myapp-data/host-file.txt
(base) khushiyadave@Khushis-MacBook-Air-2 ~ % docker exec web3 cat /app/data/host-file.txt
From Host
(base) khushiyadave@Khushis-MacBook-Air-2 ~ % docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
87affc815acd7fd2b2d23b8adaf58ca337e9bb92896af1375c742b5aaaab69n
```

22.6

22.6.1 Lab 3: Practical Volume Examples

22.6.1.1 Step 7 — MySQL with Persistent Volume

```
docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
```

Remove the container

```
docker stop mysql-db && docker rm mysql-db
```

Recreate with same volume - data is preserved!

```
docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
```

```
docker ps
```



```
(base) ksh@iMacBook-Pro:~/php7.4-MacBook-Air-2 % % docker run -d \
--name mysql-db \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:5.7
827f7f615ad75d702027848d8f58ca33e3995099a1375c752b68a8969
(base) ksh@iMacBook-Pro:~/php7.4-MacBook-Air-2 % % docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS               NAMES
827f7f615ad7        mysql:5.7           "docker-entrypoint..." 22 seconds ago     Up 22 seconds      3306/tcp, 33060/tcp  mysql-db
853709e67f8d2       nginx               "docker-entrypoint..." About a minute ago  Up About a minute  80/tcp              web3
82397793e72         nginx               "docker-entrypoint..." 3 minutes ago      Up 3 minutes       80/tcp              web2
8ff634732c38        nginx               "docker-entrypoint..." 4 minutes ago      Up 4 minutes       80/tcp              web1
8447072e771a        wordpress:latest    "docker-entrypoint..." 2 weeks ago        Up 11 minutes      80/tcp              docker-compose-demo-wordpress-1
8ed1968079e        wordpress:latest    "docker-entrypoint..." 2 weeks ago        Up 11 minutes      80/tcp              docker-compose-demo-wordpress-2
332df1e198d         mysql:5.7           "docker-entrypoint..." 2 weeks ago        Up 11 minutes      3306/tcp, 33060/tcp  mysql
(base) ksh@iMacBook-Pro:~/php7.4-MacBook-Air-2 % % docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS               NAMES
827f7f615ad7        mysql:5.7           "docker-entrypoint..." 2 minutes ago      Up 2 minutes       3306/tcp, 33060/tcp  mysql-db
853709e67f8d2       nginx               "docker-entrypoint..." 3 minutes ago      Up 3 minutes       80/tcp              web3
82397793e72         nginx               "docker-entrypoint..." 5 minutes ago      Up 5 minutes       80/tcp              web2
8ff634732c38        nginx               "docker-entrypoint..." 7 minutes ago      Up 7 minutes       80/tcp              web1
8447072e771a        wordpress:latest    "docker-entrypoint..." 2 weeks ago        Up 13 minutes      80/tcp              docker-compose-demo-wordpress-1
8ed1968079e        wordpress:latest    "docker-entrypoint..." 2 weeks ago        Up 13 minutes      80/tcp              docker-compose-demo-wordpress-2
332df1e198d         mysql:5.7           "docker-entrypoint..." 2 weeks ago        Up 13 minutes      3306/tcp, 33060/tcp  mysql
(base) ksh@iMacBook-Pro:~/php7.4-MacBook-Air-2 % % docker stop mysql-db
mysql-db
mysql-db
(base) ksh@iMacBook-Pro:~/php7.4-MacBook-Air-2 % % docker run -d \
--name new-mysql \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:5.7
8c8a2154ea338f80665529c79851be85686e52948ec448274ef28ee99c3
(base) ksh@iMacBook-Pro:~/php7.4-MacBook-Air-2 % % docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS               NAMES
8c8a2154ea33        mysql:5.7           "docker-entrypoint..." 7 seconds ago      Up 7 seconds       3306/tcp, 33060/tcp  new-mysql
853709e67f8d2       nginx               "docker-entrypoint..." 4 minutes ago      Up 4 minutes       80/tcp              web3
82397793e72         nginx               "docker-entrypoint..." 6 minutes ago      Up 6 minutes       80/tcp              web2
8ff634732c38        nginx               "docker-entrypoint..." 7 minutes ago      Up 7 minutes       80/tcp              web1
8447072e771a        wordpress:latest    "docker-entrypoint..." 2 weeks ago        Up 14 minutes      80/tcp              docker-compose-demo-wordpress-1
8ed1968079e        wordpress:latest    "docker-entrypoint..." 2 weeks ago        Up 14 minutes      80/tcp              docker-compose-demo-wordpress-2
332df1e198d         mysql:5.7           "docker-entrypoint..." 2 weeks ago        Up 14 minutes      3306/tcp, 33060/tcp  mysql
```

22.7

22.7.0.1 Step 8 — Nginx with Config Bind Mount

```
mkdir ~/nginx-config
```

```
echo 'server {
    listen 80;
    server_name localhost;
    location / {
        return 200 "Hello from mounted config!";
    }
}' > ~/nginx-config/nginx.conf
```

```
docker run -d \
--name nginx-custom \
-p 8080:80 \
-v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
nginx
```

```
curl http://localhost:8080
```

Output: Hello from mounted config!

```
(base) khushiyadave@Khushis-MacBook-Air-2 ~ % mkdir ~/nginx-config
(base) khushiyadave@Khushis-MacBook-Air-2 ~ % echo 'server {
  listen 80;
  server_name localhost;
  location / {
    return 200 "Hello from mounted config!";
  }
}' > ~/nginx-config/nginx.conf
(base) khushiyadave@Khushis-MacBook-Air-2 ~ % docker run -d \
  --name nginx-custom \
  -p 8080:80 \
  -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
  nginx
1a8a47eb558c657686ad247e0e1652a92359d4dba7b5044dcd12f4f1d4b6a66e
(base) khushiyadave@Khushis-MacBook-Air-2 ~ % curl http://localhost:8080
Hello from mounted config!%
```

22.8

22.8.1 Lab 4: Volume Management Commands

22.8.1.1 Step 9 — Volume Management

```
docker volume ls           # list all volumes
docker volume create app-volume # create a volume
docker volume inspect app-volume # inspect details
docker volume prune        # remove unused anonymous volumes
docker volume rm volume-name # remove specific volume
```

```
khushiyadave@Khushis-MacBook-Air-2 ~ % docker volume ls
DRIVER      VOLUME NAME
local       8656cd6c89f184e4e895d0b2469679b92986d870d95d17f5af70bfac78a5f777
local       docker-compose-demo_mysql_data
local       docker-compose-demo_wp_content
local       mydata
local       mysql-data
(base) khushiyadave@Khushis-MacBook-Air-2 ~ % docker volume create app-volume
app-volume
(base) khushiyadave@Khushis-MacBook-Air-2 ~ % docker volume inspect app-volume
[
  {
    "CreatedAt": "2026-03-14T17:48:24Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
    "Name": "app-volume",
    "Options": null,
    "Scope": "local"
  }
]
(base) khushiyadave@Khushis-MacBook-Air-2 ~ % docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
```

22.9

22.10 Part 2: Environment Variables

Environment variables allow dynamic configuration of containers without hardcoding values into the image.

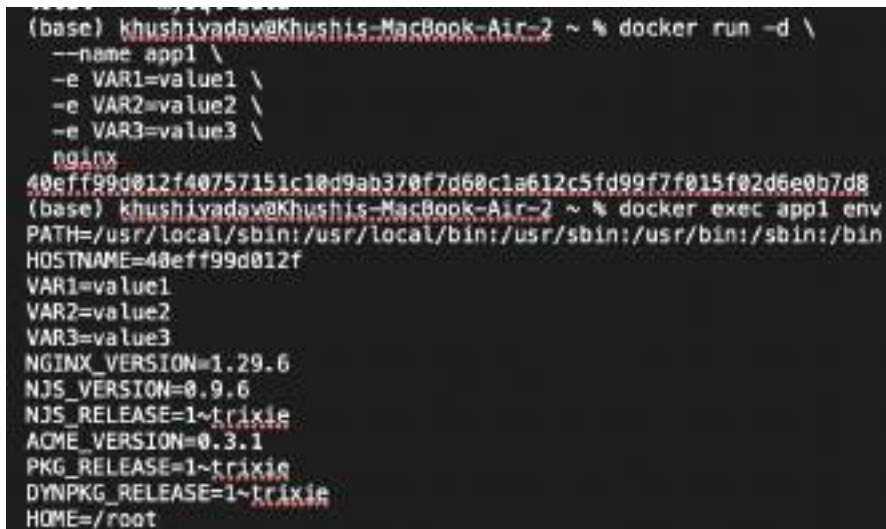
22.10.1 Lab 1: Setting Environment Variables

22.10.1.1 Step 10 — Using the -e Flag

```
docker run -d \  
  --name app1 \  
  -e VAR1=value1 \  
  -e VAR2=value2 \  
  -e VAR3=value3 \  
  nginx
```

```
docker exec app1 env
```

Output includes: VAR1=value1, VAR2=value2, VAR3=value3



```
(base) khushiyadawa@Khushis-MacBook-Air-2 ~ % docker run -d \  
  --name app1 \  
  -e VAR1=value1 \  
  -e VAR2=value2 \  
  -e VAR3=value3 \  
  nginx  
40eff99d012f40757151c10d9ab370f7d60c1a612c5fd99f7f015f02d6e0b7d8  
(base) khushiyadawa@Khushis-MacBook-Air-2 ~ % docker exec app1 env  
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin  
HOSTNAME=40eff99d012f  
VAR1=value1  
VAR2=value2  
VAR3=value3  
NGINX_VERSION=1.29.6  
NJS_VERSION=0.9.6  
NJS_RELEASE=1~trixie  
ACME_VERSION=0.3.1  
PKG_RELEASE=1~trixie  
DYNPKG_RELEASE=1~trixie  
HOME=/root
```

22.11

22.11.0.1 Step 11 — Using -env-file

```
echo "DATABASE_HOST=localhost" > ~/.env  
echo "DATABASE_PORT=5432" >> ~/.env  
echo "API_KEY=secret123" >> ~/.env
```

```
docker run -d \  
  --name app2 \  
  --env-file ~/.env \  
  nginx
```

```
docker exec app2 env
```

Output includes: DATABASE_HOST, DATABASE_PORT, API_KEY

```
(base) khushivadava@Khushis-MacBook-Air-2 ~ % cd ~ && echo "DATABASE_HOST=localhost" > .env
echo "DATABASE_PORT=5432" >> .env
echo "API_KEY=secret123" >> .env
(base) khushivadava@Khushis-MacBook-Air-2 ~ % cat ~/.env
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
(base) khushivadava@Khushis-MacBook-Air-2 ~ % docker run -d \
  --name app2 \
  --env-file ~/.env \
  nginx
402c81084d27024f99a8c8fa24bf69c0783ab907726625c7f6ae8eda03f87c60
(base) khushivadava@Khushis-MacBook-Air-2 ~ % docker exec app2 env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=402c81084d27
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
NGINX_VERSION=1.29.6
NJS_VERSION=0.9.6
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
HOME=/root
```

22.12

22.12.1 Lab 2: Flask App with Environment Variables

22.12.1.1 Step 12 — Create Flask Application app.py:

```
import os
from flask import Flask
app = Flask(__name__)

db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

requirements.txt:

```
flask
```

Dockerfile:

```
FROM python:3.9-slim
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1
WORKDIR /app
COPY requirements.txt .
```

```

(base) khushiyadav@Khushis-MacBook-Air-2 ~ % mkdir ~/flask-app && cd ~/flask-app
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % cat > app.py << 'EOF'
import os
from flask import Flask
app = Flask(__name__)

db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
EOF
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % cat ~/flask-app/app.py
import os
from flask import Flask
app = Flask(__name__)

db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'
api_key = os.environ.get('API_KEY')

@app.route('/config')
def config():
    return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)

```

Figure 20: screenshot 2


```

RUN pip install -r requirements.txt
COPY app.py .
ENV PORT=5000
ENV DEBUG=false
EXPOSE 5000
CMD ["python", "-u", "app.py"]

```

```
docker build -t flask-app .
```

```

(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % echo "flask" > requirements.txt
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % cat > Dockerfile << 'EOF'
FROM python:3.9-slim
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
ENV PORT=5000
ENV DEBUG=false
EXPOSE 5000
CMD ["python", "app.py"]
EOF
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker build -t flask-app .
[+] Building 19.3s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring Dockerfile: 257B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/python:3.9-slim@sha256:2d97f6918b18bd338d3868f261f53f144965f755399a81ecda1e13cf1731b19
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6918b18bd338d3868f261f53f144965f755399a81ecda1e13cf1731b19
=> => sha256:c25f4b0d147388e81e1a1d95087669c78e4451082a128c2188c1655794306 251B / 251B
=> => sha256:47798ed0cc3edbebd2e4918dfbec45228c5a11d71738914c68455707bf9124c 1.27MB / 1.27MB
=> => sha256:ab7c704fa8c581a81a5aa6d1c1e2558e4b985d3ce3827b8ed869447b8c51129 13.83MB / 13.83MB
=> => sha256:a18e51132678581ec8159c78a89ee0f72af54687fc7263d419ce21b8366147 38.14MB / 38.14MB
=> => extracting sha256:a18e51132678581ec8159c78a89ee0f72af54687fc7263d419ce21b8366147
=> => extracting sha256:47798ed0cc3edbebd2e4918dfbec45228c5a11d71738914c68455707bf9124c
=> => extracting sha256:ab7c704fa8c581a81a5aa6d1c1e2558e4b985d3ce3827b8ed869447b8c51129
=> => extracting sha256:c25f4b0d147388e81e1a1d95087669c78e4451082a128c2188c1655794306
=> [internal] load build context
=> => transferring context: 578B
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install -r requirements.txt
=> [5/5] COPY app.py .
=> => exporting layers
=> => exporting manifest sha256:f807cc9c3ed1508ead8f6e6cd21ec4c18a519a10a7edec697516ce5d8dd8a9
=> => exporting config sha256:97e018f8e318d13c2865ad1cc21bc57b4f1011be178a508eb21ed1ade97
=> => exporting attestation manifest sha256:c664c72d6644b778a5d6e02168c85d70c713b865b78b1ee3f1b5e53bbcb821d
=> => exporting manifest list sha256:3b181d842f9729e4043845713715b94638685c27c86d755635f4e16217dc3d56
=> => saving to docker.io/library/flask-app:latest
=> => unpacking to docker.io/library/flask-app:latest
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker run -d \
  --name flask-app \
  -p 5000:5000 \
  -e DATABASE_HOST="prod-db.example.com" \
  -e DEBUG="true" \
  -e API_KEY="example123" \
  flask-app
858d3c738e91d59a251481e91c7c3a1004fa7e2473229a191aef58535f85f5

```

22.12.1.2 Error: Container exited immediately in detached mode

docker logs flask-app → (empty output)

docker ps | grep flask-app → (no output, container not running)

Cause: Python output buffering — Flask was not emitting logs, so the container appeared to crash silently.

22.12.1.3 Fix: Use -u flag for unbuffered output Changed CMD in Dockerfile:

```
CMD ["python", "-u", "app.py"]
```

Then rebuilt the image:


```
docker build -t flask-app .
```

22.12.1.4 Error: Port 5000 already in use

Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:5000
→ 127.0.0.1:0: listen tcp 0.0.0.0:5000: bind: address already in use

Cause: macOS uses port 5000 for AirPlay Receiver by default.

22.12.1.5 Fix: Use port 5001 instead

```
docker run -d \  
  --name flask-app \  
  -p 5001:5000 \  
  -e DATABASE_HOST="prod-db.example.com" \  
  -e DEBUG="true" \  
  -e API_KEY="myapikey123" \  
  flask-app
```

22.12.1.6 Step 13 — Test Flask App

```
curl http://localhost:5001/config
```

Response:

```
{  
  "db_host": "prod-db.example.com",  
  "debug": true,  
  "has_api_key": true  
}
```

```

=> exporting to image
=> exporting layers
=> exporting manifest sha256:b84d06cb96f44cd219c2d3c6eb68f86e7b67d893d1989e327a5745916815
=> exporting config sha256:26098738398f1d55684b2e174d7a48c6c53153e180cc1c18491c5c6b98260
=> exporting attestation manifest sha256:1bb594731deb1e484871beafcd141d0f8da989ca913366a74dca0e1fa887a0b
=> exporting manifest list sha256:ca0e0450a10032093c1950097cc46835893e73c1681c56d132f440e519f88d
=> naming to docker.io/library/flask-app:latest
=> unpacking to docker.io/library/flask-app:latest
(base) khshisyndav@Khushis-MacBook-Air-2 flask-app % docker run -d \
  --name flask-app \
  -p 5000:5000 \
  -e DATABASE_HOST="prod-db.example.com" \
  -e DEBUG="true" \
  -e API_KEY="myapikey123" \
  flask-app
99a0a6036f70cc0f548f8a3a2a4cd632146df25f9f915f09462679c35809cc
docker: Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:5000 -> 127.0.0.1:0: listen tcp 0.0.0.0:5000: bind: address already in use.

Run 'docker run --help' for more information
(base) khshisyndav@Khushis-MacBook-Air-2 flask-app % docker rm flask-app
flask-app
(base) khshisyndav@Khushis-MacBook-Air-2 flask-app % docker run -d \
  --name flask-app \
  -p 5001:5000 \
  -e DATABASE_HOST="prod-db.example.com" \
  -e DEBUG="true" \
  -e API_KEY="myapikey123" \
  flask-app
c94cd32cf1924f300a8346182d093bce81d9c5dc18938f38ff0a2c0b0a826024
(base) khshisyndav@Khushis-MacBook-Air-2 flask-app % docker logs flask-app
* Serving Flask app "app"
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.9:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 185-668-530
(base) khshisyndav@Khushis-MacBook-Air-2 flask-app % curl http://localhost:5001/config
{
  "db_host": "prod-db.example.com",
  "debug": true,
  "has_api_key": true
}

```

22.13

22.14 Part 3: Docker Monitoring

Docker provides several commands to monitor container health, resource usage, processes, and logs.

22.14.1 Lab 1: Resource Metrics

22.14.1.1 Step 14 — docker stats

Snapshot of all containers

```
docker stats --no-stream
```

Formatted output

```
docker stats --no-stream --format \
  "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.NetIO}}"
```

```
(base) khs@huyadav02-travis-MacBook-Air:~$ flask-app % docker stats --no-stream
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT     MEM %     NET I/O     BLOCK I/O     PIDS
e98ad32ef162   flask-app 0.23%     43.34MiB / 3.827GiB    1.11%     1.55kB / 242B    0B / 8B       3
a80c21884627   app2      0.00%     7.588MiB / 3.827GiB    0.19%     1.46kB / 126B    0B / 12.3kB    9
48eff99a812f   app1      0.00%     7.414MiB / 3.827GiB    0.19%     1.56kB / 126B    0B / 12.3kB    9
1a8a47e8558c   flask-app 0.00%     7.531MiB / 3.827GiB    0.19%     2.55kB / 816B    0B / 4.1kB     9
b2ba212646a2   new-mysql 0.01%     367.4MiB / 3.827GiB    9.38%     1.88kB / 126B    65.5kB / 15.2MB 37
8ffc70b9dfe2   web3      0.00%     7.473MiB / 3.827GiB    0.19%     2.13kB / 126B    69.8kB / 12.3kB 9
b80397f93df2   web2      0.00%     7.588MiB / 3.827GiB    0.19%     2.26kB / 126B    0B / 12.3kB    9
8ff634732c70   web1      0.00%     9.613MiB / 3.827GiB    0.25%     2.56kB / 126B    2.37MB / 12.3kB 0
9467af2ef71a   docker-compose-down-redis-1 0.01%     41.04MiB / 512MiB     8.01%     2kB / 126B      27.7MB / 4.1kB  0
6ed19a58279e   docker-compose-down-redis-2 0.00%     48.85MiB / 512MiB     7.98%     2kB / 126B      26.9MB / 4.1kB  6
332df15e19bd   mysql     0.22%     415.1MiB / 3.827GiB   10.50%     2kB / 126B      76.9MB / 17MB   37

(base) khs@huyadav02-travis-MacBook-Air:~$ flask-app % docker stats --no-stream --format "table {{.Name}}\t{{.CpuPerc}}\t{{.MemUsage}}\t{{.NetIO}}"
NAME           CPU %     MEM USAGE / LIMIT     NET I/O
flask-app      0.23%     43.34MiB / 3.827GiB    1.55kB / 242B
app2           0.00%     7.588MiB / 3.827GiB    1.46kB / 126B
app1           0.00%     7.414MiB / 3.827GiB    1.56kB / 126B
flask-app      0.00%     7.531MiB / 3.827GiB    2.55kB / 816B
new-mysql      0.01%     367.4MiB / 3.827GiB    1.88kB / 126B
web3           0.00%     7.473MiB / 3.827GiB    2.13kB / 126B
web2           0.00%     7.588MiB / 3.827GiB    2.26kB / 126B
web1           0.00%     9.613MiB / 3.827GiB    2.56kB / 126B
docker-compose-down-redis-1 0.01%     41.04MiB / 512MiB     2kB / 126B
docker-compose-down-redis-2 0.01%     48.85MiB / 512MiB     2kB / 126B
mysql          0.22%     415.1MiB / 3.827GiB    2kB / 126B
```

22.15

22.15.0.1 Step 14b — docker top

docker top flask-app

Output:

```
UID      PID      PPID     CMD
root     2448     2425     python -u app.py
root     2469     2448     /usr/local/bin/python /app/app.py
```

```
(base) khs@huyadav02-travis-MacBook-Air:~$ flask-app % docker top flask-app
UID      PID      PPID     CPU %     TIME
root     2448     2425     0         00:00
root     2469     2448     0         00:00
CMD
python -u app.py
/usr/local/bin/python /app/app.py
```

22.16

22.16.1 Lab 2: Log Monitoring

22.16.1.1 Step 15 — docker logs

```
docker logs flask-app           # view logs
docker logs -t flask-app        # with timestamps
docker logs --tail 5 flask-app   # last 5 lines
docker logs -f --tail 10 -t flask-app # follow live logs
```

Live log output showed: - Flask startup on port 5000 - 3 GET /config requests with status 200

22.19.1 Lab 1: Network Types

22.19.1.1 Step 17 — List and Create Networks

```
docker network ls
```

Output: bridge, host, none (defaults)

```
docker network create my-network
```

```
docker network inspect my-network
```

```
(base) khushiyadav@khushis-MacBook-Air-2 flask-app % docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
f484b063b6a8        bridge              bridge              local
27941e90f3c5        docker-compose-demo_default    bridge              local
1cb66430c18a2        host                host                local
9297a580e894        none                null                local
(base) khushiyadav@khushis-MacBook-Air-2 flask-app % docker network create my-network
45870714af0966b8c3aac31eb6c5eb8edb7eb549d4ebd29a2c3cfe411b4dadd
(base) khushiyadav@khushis-MacBook-Air-2 flask-app % docker network inspect my-network
[
  {
    "Name": "my-network",
    "Id": "45870714af0966b8c3aac31eb6c5eb8edb7eb549d4ebd29a2c3cfe411b4dadd",
    "Created": "2026-03-14T18:16:40.143954879Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.19.0.0/16",
          "IPRange": "",
          "Gateway": "172.19.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {},
    "Containers": {},
    "Status": {
      "IPAM": {
        "Subnets": {
          "172.19.0.0/16": {
            "IPsInUse": 3,
            "DynamicIPsAvailable": 65533
          }
        }
      }
    }
  }
]
```

22.20

22.20.0.1 Step 17b — Container Communication via Custom Network

```
docker run -d --name net-web1 --network my-network nginx
```

```
docker run -d --name net-web2 --network my-network nginx
```

Test communication using container name as hostname

```
docker exec net-web1 curl -s http://net-web2
# Output: nginx welcome HTML page
![screenshot 2 ](/5.20.png)
```

> Docker's built-in DNS resolved `net-web2` to its container IP automatically.

Lab 2: Network Isolation

Step 18 - None Network (Full Isolation)

```
```bash
docker run -d --name isolated-app --network none alpine sleep 3600
docker exec isolated-app ifconfig
Only loopback (lo) interface visible - no network access
```

```
(base) shushiyadav@Shushis-MacBook-Air-2 flask-app % docker run -d --name isolated-app --network none alpine sleep 3600
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
c8ad6cd72600: Pull complete
37893446b8e8: Download complete
c094f79e6e08: Download complete
Digest: sha256:25189184c715dad752c6312a8623239686a3a2871e0825f29e6b8f219d31f659
Status: Downloaded newer image for alpine:latest
954a583385291680c115f1ebac7c7688086e04542a7434c6eb997a887fc1712
(base) shushiyadav@Shushis-MacBook-Air-2 flask-app % docker exec isolated-app ifconfig
lo
 Link encap:Local Loopback
 inet addr:127.0.0.1 Mask:255.0.0.0
 inet6 addr: ::1/128 Scope:Host
 UP LOOPBACK RUNNING MTU:65536 Metric:1
 RX packets:0 errors:0 dropped:0 overruns:0 frame:0
 TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
 collisions:0 txqueuelen:1000
 RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

(base) shushiyadav@Shushis-MacBook-Air-2 flask-app % docker exec net-web1 nslookup net-web2
OCI runtime exec failed: exec failed: unable to start container process: exec: "nslookup": executable file not found in $PATH
```

## 22.21

### 22.21.0.1 Error: nslookup not found in nginx image

OCI runtime exec failed: exec: "nslookup": executable file not found in \$PATH

### 22.21.0.2 Fix: Use docker network inspect instead

```
docker network inspect my-network
```

### 22.21.0.3 Error: ping not found in nginx image

OCI runtime exec failed: exec: "ping": executable file not found in \$PATH

### 22.21.0.4 Fix: Connectivity already proven via curl Container communication was already confirmed using:

```
docker exec net-web1 curl -s http://net-web2
```

### 22.21.0.5 Step 18b — Network Inspection

```
docker network inspect my-network
```

Output confirmed:



```

(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker run -d --name host-app --network host oslokg
a98499355968ff6356d1a7a2e5a7198c8c4791576edc72c8a3579e87b01829bf
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker run -d --name isolated-app --network none alpine sleep 3600
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
c8ad8cd72800: Pull complete
37843440b0e8: Download complete
cb94172ef0e8: Download complete
Digest: sha256:25189184c71bdad752c6312a8623239966e9a2871e0825f29ecb8f2190c3f429
Status: Downloaded newer image for alpine:latest
054a583385291689c115f1ebac7688086e94542a7434c6ab097a0887fc1712
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker exec isolated-app ifconfig
lo
 Link encap:Local Loopback
 inet addr:127.0.0.1 Mask:255.0.0.0
 inet6 addr: ::1/128 Scope:Host
 UP LOOPBACK RUNNING MTU:65536 Metric:1
 RX packets:0 errors:0 dropped:0 overruns:0 frame:0
 TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
 collisions:0 txqueuelen:1000
 RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker exec net-web1 oslokgp net-web2
OCI runtime exec failed: exec failed: unable to start container process: exec: "oslokgp": executable file not found in $PATH

```

Figure 21: screenshot 2

```

net-web1 → 172.19.0.2/16
net-web2 → 172.19.0.3/16
Gateway → 172.19.0.1
Subnet → 172.19.0.0/16

```

### 22.21.0.6 Step 18c — Connect Existing Container to Network

```

docker network connect my-network flask-app
docker network inspect my-network
flask-app now appears in Containers section

```

```
(base) shushiyang@Shushis-MacBook-Air-2 flask-app % docker exec net-web1 nslookup net-web2
OCI runtime exec failed: exec failed: unable to start container process: exec: "nslookup": executable file not found in $PATH
(base) shushiyang@Shushis-MacBook-Air-2 flask-app % docker exec net-web1 ping -c 3 net-web2
OCI runtime exec failed: exec failed: unable to start container process: exec: "ping": executable file not found in $PATH
(base) shushiyang@Shushis-MacBook-Air-2 flask-app % docker network inspect my-network
{
 "Name": "my-network",
 "Id": "45870714afc366b8c3a0c31eb6c5eb8ed07eb549d4c6d29a2c3c0fed11b4d0dd",
 "Created": "2020-03-14T18:16:40.143954879Z",
 "Scope": "local",
 "Driver": "bridge",
 "EnableIPv4": true,
 "EnableIPv6": false,
 "IPAM": {
 "Driver": "default",
 "Options": {},
 "Config": {
 {
 "Subnet": "172.19.0.0/16",
 "IPRange": "",
 "Gateway": "172.19.0.1"
 }
 }
 },
 "Internal": false,
 "Attachable": false,
 "Ingress": false,
 "ConfigFrom": {
 "Network": ""
 },
 "ConfigOnly": false,
 "Options": {
 "com.docker.network.enable_ipv4": "true",
 "com.docker.network.enable_ipv6": "false"
 },
 "Labels": {},
 "Containers": {
 "6230cc36a218a5936f6b0c331b0b6fc64923b991830670304a0ba2c47775582a": {
 "Name": "net-web2",
 "EndpointID": "bc297c18a3c23c8469d0d8174e6e88cc24a0899cd8e2ec842da2f27f0b663a9c6",
 "MacAddress": "cc:22:4d:56:2f:92",
 "IPv4Address": "172.19.0.3/16",
 "IPv6Address": ""
 },
 "63c30d32ca9727f92ef4e781443536cbf05d43b48705c08cd1d122081e675ed0": {
 "Name": "net-web1",
 "EndpointID": "dc64d9a9e0cb3afbd431b2b7da769091ab711647ad34cf332045763c2f14006d",
 "MacAddress": "c2:d2:08:ac:5d:d1",
 "IPv4Address": "172.19.0.2/16",
 "IPv6Address": ""
 }
 },
 "Status": {
 "IPAM": {
 "Subnets": {
 "172.19.0.0/16": {
 "IPsInUse": 5,
 "DynamicIPsAvailable": 65531
 }
 }
 }
 }
}
```

22.22

## 22.23 Part 5: Real-World Example

A full 3-tier application stack: **Flask** web app + **PostgreSQL** database + **Redis** cache — all connected via a custom Docker network with persistent volumes.

### 22.23.1 Architecture

myapp-network

flask-app	postgres	redis
:5001→5000	:5432	:6379

volume

```
postgres-data redis-data
```

---

## 22.23.2 Step 19 — Deploy the Stack

### 22.23.2.1 Create Network

```
docker network create myapp-network
```

### 22.23.2.2 Start PostgreSQL

```
docker run -d \
 --name postgres \
 --network myapp-network \
 -e POSTGRES_PASSWORD=mysecretpassword \
 -e POSTGRES_DB=mydatabase \
 -v postgres-data:/var/lib/postgresql/data \
 postgres:15
```

### 22.23.2.3 Start Redis

```
docker run -d \
 --name redis \
 --network myapp-network \
 -v redis-data:/data \
 redis:7-alpine
```

### 22.23.2.4 Verify Services

```
docker ps | grep -E "postgres|redis"
```

Output:

```
redis:7-alpine → Up (port 6379)
postgres:15 → Up (port 5432)
```

## 22.24



## 22.24.1 Step 20 — Connect Flask and Verify Network

```
docker network connect myapp-network flask-app
docker network inspect myapp-network | grep "Name"
```

---

### 22.24.1.1 Error: curl not found in flask image

OCI runtime exec failed: exec: "curl": executable file not found in \$PATH

Cause: python:3.9-slim is a minimal image — no curl installed.

### 22.24.1.2 Fix: Use docker network inspect

```
docker network inspect myapp-network
Confirmed flask-app, postgres, and redis all listed under Containers
```

---

```

(base) shushiyang@Shushis-MacBook-Air-2 flask-app % docker network create myapp-network
8f8c98db7e0a4cb15c14c8a7ca9e66d17df9291898816fc517d2c324d8fa9ef
(base) shushiyang@Shushis-MacBook-Air-2 flask-app % docker run -d \
 --name postgres \
 --network myapp-network \
 -e POSTGRES_PASSWORD=secretpassword \
 -e POSTGRES_DB=mydatabase \
 -v postgres-data:/var/lib/postgresql/data \
 postgres:15
Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
39a06368a311: Pull complete
331a896bfc4f: Pull complete
48ffcc14736fe: Pull complete
79967bed2ef2: Pull complete
23e9a8f949ce: Pull complete
736f3fee7418: Pull complete
07feb1338a04: Pull complete
41337a8e8550: Pull complete
606ff205986c: Pull complete
ce71be38ed44: Pull complete
8d332802d738: Pull complete
80df52df9af7: Pull complete
d2893ff13806f: Pull complete
8f9479a8dcb6: Download complete
b08d682aef: Download complete
Digest: sha256:f38c3de9ac9cc938dc627ef7231699867c694b5f349fadb924c8c377428c399
Status: Downloaded newer image for postgres:15
dfba5e66a88a759af64b72d9181f30ed0eebfcc595546ec5d8fd2c94fc191362
(base) shushiyang@Shushis-MacBook-Air-2 flask-app % docker run -d \
 --name redis \
 --network myapp-network \
 -v redis-data:/data \
 redis:7-alpine
Unable to find image 'redis:7-alpine' locally
7-alpine: Pulling from library/redis
b391ede473c1: Pull complete
246d55128559: Pull complete
f2eae7365acd: Pull complete
a447a5de8f4e: Pull complete
4f4fb708ef54: Pull complete
af7a28e28324: Pull complete
b6fd4f7e9d8b: Pull complete
9ca97655d7b1: Pull complete
424c68a579b: Download complete
34ac65268e22: Download complete
Digest: sha256:8b81dd37f7827bec4c516d41acfb9fc24688789c6d4a4579a20c5b9c53d7963
Status: Downloaded newer image for redis:7-alpine
733a8dc54b3d55e770663ab81a8fc8fc4d061c4212031d418e481f820426c8e

```

Figure 22: screenshot 2

```

(base) shushiyang@Shushis-MacBook-Air-2 flask-app % docker network connect myapp-network flask-app
(base) shushiyang@Shushis-MacBook-Air-2 flask-app % docker exec flask-app curl -s telnet://postgres:5432 || echo "Port reachable!"
could not connect to postgres:5432: Connection refused
(base) shushiyang@Shushis-MacBook-Air-2 flask-app % docker exec flask-app curl -s --max-time 2 http://postgres:5432 || echo "Postgres reachable!"
could not connect to postgres:5432: Connection refused
(base) shushiyang@Shushis-MacBook-Air-2 flask-app % docker exec flask-app curl -s --max-time 2 http://postgres:5432
could not connect to postgres:5432: Connection refused
8CC runtime exec failed: exec failed: unable to start container process: exec: "curl": executable file not found in $PATH
(base) shushiyang@Shushis-MacBook-Air-2 flask-app % docker network inspect myapp-network | grep "Name"
 "Name": "myapp-network",
 "Name": "redis",
 "Name": "postgres",
 "Name": "flask-app",

```

Figure 23: screenshot 2

### 22.24.2 Step 20b — Verify Logs

```
docker logs postgres
PostgreSQL 15 initialized, mydatabase created
"database system is ready to accept connections"
```

```
docker logs redis
Redis 7.4.8 started on port 6379
"Ready to accept connections tcp"
```

---

### 22.24.3 Step 21 — Monitor All Services

```
docker stats --no-stream --format \
"table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
```

```
(base) [root@kvm01 ~]# docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}
NAME CPU % MEM USAGE / LIMIT
redis 1.01% 9.12MiB / 3.82GiB
postgres 0.20% 28.79MiB / 3.82GiB
isolated-app 0.00% 336KiB / 3.82GiB
test-app 0.00% 3.27MiB / 3.82GiB
test-app2 0.00% 2.47MiB / 3.82GiB
test-app3 0.00% 2.49MiB / 3.82GiB
flask-app 0.24% 45.39MiB / 3.82GiB
app2 0.00% 2.38MiB / 3.82GiB
app3 0.00% 2.45MiB / 3.82GiB
redis-cache 0.00% 2.33MiB / 3.82GiB
redis 0.00% 167.4MiB / 3.82GiB
web1 0.00% 2.42MiB / 3.82GiB
web2 0.00% 2.38MiB / 3.82GiB
web3 0.00% 3.61MiB / 3.82GiB
docker-00000000-0000-0000-0000-000000000000 0.01% 45.84MiB / 512MiB
docker-00000000-0000-0000-0000-000000000000 0.01% 48.85MiB / 512MiB
mysql 0.50% 405.1MiB / 3.82GiB

(base) [root@kvm01 ~]# docker logs postgres
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.

The database cluster will be initialized with locale "en_US.utf8".
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

fixing permissions on existing directory /var/lib/postgresql/data ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default time zone ... Etc/UTC
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok

(base) [root@kvm01 ~]# docker logs redis
1: 1: 24 Mar 2025 18:25:00.588 * Redis 7.4.8 started on port 6379. Ready to accept connections tcp
1: 1: 24 Mar 2025 18:25:00.588 * Redis 7.4.8 started on port 6379. Ready to accept connections tcp
1: 1: 24 Mar 2025 18:25:00.588 * Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
1: 1: 24 Mar 2025 18:25:00.588 * Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
1: 1: 24 Mar 2025 18:25:00.587 * Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
1: 1: 24 Mar 2025 18:25:00.587 * Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
1: 1: 24 Mar 2025 18:25:00.587 * Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
1: 1: 24 Mar 2025 18:25:00.587 * Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
1: 1: 24 Mar 2025 18:25:00.587 * Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
1: 1: 24 Mar 2025 18:25:00.587 * Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
1: 1: 24 Mar 2025 18:25:00.587 * Warning: no config file specified, using the default config. In order to specify a config file use redis-server /path/to/redis.conf
```

## 22.25 Cleanup

```
Stop and remove all containers
```

```
docker stop $(docker ps -aq)
```

```
docker rm $(docker ps -aq)
```

```
Remove unused volumes (anonymous only - named volumes are kept)
```

```
docker volume prune -f
```

```
Remove unused networks (except defaults)
```

```
docker network prune -f
```

```
Remove unused images
```

`docker image prune -f`

```

(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker stop $(docker ps -aq)
733a0de54b3d
dfba5e66a08a
954a58330529
a08499395968
623bce36a218
d3c38d32ca97
e94ad32eff92
402c81084d27
40eff99d012f
1a8a47eb558c
be8a21564ea3
0fc7bbe0fd62
b02897f93df2
8ff634732c70
5c6eee0690d4
9447bf2ef71a
6ed19b50279e
332df11e19bd
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker rm $(docker ps -aq)
733a0de54b3d
dfba5e66a08a
954a58330529
a08499395968
623bce36a218
d3c38d32ca97
e94ad32eff92
402c81084d27
40eff99d012f
1a8a47eb558c
be8a21564ea3
0fc7bbe0fd62
b02897f93df2
8ff634732c70
5c6eee0690d4
9447bf2ef71a
6ed19b50279e
332df11e19bd
```



```

352d11c1900
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker volume prune -f
Deleted Volumes:
8656cd6c89f184e4e895d0b2469679b92906d870d95d17f5af70bfac78a5f777

Total reclaimed space: 0B
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker network prune -f
Deleted Networks:
myapp-network
docker-compose-demo_default
my-network

(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker image prune -f
Total reclaimed space: 0B
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app % docker ps -a
docker volume ls
docker network ls
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
DRIVER VOLUME NAME
local app-volume
local docker-compose-demo_mysql_data
local docker-compose-demo_wp_content
local mydata
local mysql-data
local postgres-data
local redis-data
NETWORK ID NAME DRIVER SCOPE
f484b063b6a8 bridge bridge local
1eb6430c10a2 host host local
9297a58de8a4 none null local
(base) khushiyadav@Khushis-MacBook-Air-2 flask-app %

```

**After cleanup:** - Containers → 0 remaining - Networks → only **bridge**, **host**, **none** - Named volumes retained (prune never removes named volumes)

## 22.26 Errors Encountered & Fixes

#	Error	Cause	Fix
1	Cannot remove running container	<code>docker rm</code> called while container was still running	Run <code>docker stop</code> first, then <code>docker rm</code>
2	Container exited immediately in detached mode	Python output buffering — Flask not emitting logs	Added <code>-u</code> flag: CMD <code>["python", "-u", "app.py"]</code> and rebuilt
3	Port 5000 already in use	macOS AirPlay Receiver occupies port 5000 by default	Used <code>-p 5001:5000</code> to map host port 5001 to container 5000
4	<code>nslookup</code> not found in nginx image	Official nginx image is minimal — no DNS tools installed	Used <code>docker network inspect</code> to verify container IPs
5	<code>ping</code> not found in nginx image	Same reason — minimal nginx image	Connectivity already proven via <code>curl http://net-web2</code>

#	Error	Cause	Fix
6	curl not found in flask image	python:3.9-slim has no curl	Used <code>docker network inspect</code> to confirm network membership

---

## 22.27 Key Takeaways

1. **Volumes persist data** beyond container lifecycle — always use named volumes for production data
  2. **Environment variables** configure containers dynamically — use `.env` files for sensitive config
  3. **Custom networks** provide built-in DNS resolution — containers communicate using names
  4. **Monitoring commands** (`stats`, `logs`, `top`, `inspect`) help debug and optimize containers
  5. **Minimal Docker images** (nginx, python-slim) may lack common tools like curl, ping, nslookup
  6. **macOS port 5000** is reserved by AirPlay — use an alternative port when running Flask locally
  7. **docker volume prune** only removes anonymous volumes — named volumes are always preserved
  8. **Multi-container apps** should always use custom networks for isolation and service discovery
- 

## 23 Experiment 6 – Docker and Compose Exercises

### 23.1 Aim

Comparison of Docker Run and Docker Compose. `##` Objective This lab explores container fundamentals through multiple parts:

- single container deployment with docker run
  - Compose-based service definition for nginx and WordPress
  - building a custom Node.js app image
  - multi-container orchestration with persistent storage
- 

## 24 Part 1 – Single Container Comparison

This section compares running nginx directly with docker run versus using docker-compose.

### 24.1 Step 1: Run nginx with docker run

```
docker run -d \
 --name lab-nginx \
 -p 8081:80 \
 -v "${PWD}/html:/usr/share/nginx/html" \
 nginx:alpine
```

Verify the container is running:

```
docker ps
```



```

services:
 nginx:
 image: nginx:alpine
 container_name: lab-nginx
 ports:
 - "8081:80"
 volumes:
 - ./html:/usr/share/nginx/html

```

Run the Compose app:

```
docker compose up -d
```

Verify service status:

```
docker compose ps
```

Stop the service:

```
docker compose down
```

```

(base) khushiyasa@Khushis-MacBook-Air-2 sss-6 % cat > docker-compose.yml << "EOF"
services:
 nginx:
 image: nginx:alpine
 container_name: lab-nginx
 ports:
 - "8081:80"
 volumes:
 - ./html:/usr/share/nginx/html
EOF
(base) khushiyasa@Khushis-MacBook-Air-2 sss-6 % cat docker-compose.yml
services:
 nginx:
 image: nginx:alpine
 container_name: lab-nginx
 ports:
 - "8081:80"
 volumes:
 - ./html:/usr/share/nginx/html
(base) khushiyasa@Khushis-MacBook-Air-2 sss-6 % docker compose up -d
WARNING: /Users/khushiyasa/Desktop/UPES/Semester - 6/Containerization and DevOps lab/sss-6/docker-compose.yml: the attribute 'version' is obsolete, it will be confused
WARNING: No services to build
[+] up 2/2
✔ Network sss-6_default Created
✔ Container lab-nginx Created
(base) khushiyasa@Khushis-MacBook-Air-2 sss-6 % docker compose ps
WARNING: /Users/khushiyasa/Desktop/UPES/Semester - 6/Containerization and DevOps lab/sss-6/docker-compose.yml: the attribute 'version' is obsolete, it will be confused

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
lab-nginx	nginx:alpine	"/docker-entrypoint..."	nginx	33 seconds ago	Up 33 seconds	*.0.0.0:8081->80/tcp, [::]:8081->80/tcp

```

(base) khushiyasa@Khushis-MacBook-Air-2 sss-6 % docker compose down

```

Figure 25: screenshot 1

---

#  
Part  
2 –  
Multi-  
Container  
Appli-  
ca-  
tion  
(Word-  
Press  
+  
MySQL)

---

This  
sec-  
tion  
demon-  
strates  
Word-  
Press  
with  
MySQL  
using  
both  
docker  
run  
and  
docker-  
compose.  
##  
Task  
2A:  
Run  
with  
docker  
run  
Create  
a cus-  
tom  
net-  
work:  
Run  
Word-  
Press:  
“bash  
docker  
run  
-d  
—  
name  
word-  
press  
—  
network  
wp-  
net  
-p  
8082:80  
-e  
WORD-  
PRESS\_DB\_HOST=mysql  
-e  
WORD-  
PRESS\_DB\_PASSWORD=secret  
wordpress:latest  
““



Open  
in  
browser:  
##  
Task  
2B:  
Run  
with  
docker-  
compose  
Use  
the  
Com-  
pose  
file in  
Part  
B/docker-  
compose.yml:  
“‘bash  
ver-  
sion:  
‘3.8’  
services:  
mysql:  
im-  
age:  
mysql:5.7  
envi-  
ron-  
ment:  
MYSQL\_ROOT\_PASSWORD:  
secret  
MYSQL\_DATABASE:  
word-  
press  
vol-  
umes:  
-  
mysql\_data:/var/lib/mysql



---

```

wordpress:
im-
age:
word-
press:latest
ports:
-
"8082:80"
envi-
ron-
ment:
-
WORD-
PRESS_DB_HOST=mysql
-
WORD-
PRESS_DB_PASSWORD=secret
de-
pends_on:
-
mysql
volumes:
mysql_data:
""

Start
the
stack:
bash
docker
compose
up
-d
Stop
and
re-
move
vol-
umes:
##
Part
3 —
Cus-
tom
Node.js
Build
(Part
C/Task
4)

```

---

This  
part  
builds  
a sim-  
ple  
Ex-  
press  
app  
into a  
Docker  
im-  
age.  
###  
Files  
in-  
volved  
- Part  
C/Task  
4/Dock-  
erfile -  
Part  
C/Task  
4/server.js  
- Part  
C/Task  
4/pack-  
age.json  
###  
Dock-  
erfile  
“bash  
FROM  
node:18-  
alpine  
WORKDIR  
/app  
COPY  
pack-  
age\*.json  
./  
RUN  
npm  
in-  
stall  
COPY  
.  
EXPOSE  
5000  
CMD  
[“node”,  
“server.js”]  
““

---

```

###
server.js
“‘bash
const
ex-
press
= re-
quire(‘express’);
const
app
= ex-
press();
const
port
=
5000;
app.get(‘/’,
(req,
res)
=> {
res.send(‘Hello
from
your
Cus-
tom
Docker
Build!’);
});
app.listen(port,
() =>
{ con-
sole.log(Server
running
at
http://localhost:${port});
});
###
Build
and
runbash
docker
build
-t lab-
node-
app
“Part
C/Task
4”

```

---

```
docker
run
-d -p
5000:5000
```

```
name
lab-
node-
app
lab-
node-
app
““
```

```
Verify
by
open-
ing:
docker
stop
lab-
node-
app
docker
rm
lab-
node-
app
docker
image
rm
lab-
node-
app
““
```

```
##
Part
4 —
Per-
sis-
tent
MySQL
Vol-
ume
(Part
D/Task
5)
```

---

This  
task  
shows  
a  
Com-  
pose  
ser-  
vice  
with  
a  
named  
vol-  
ume  
for  
MySQL  
data  
per-  
sis-  
tence.  
###  
Files  
in-  
volved  
- Part  
D/Task  
5/docker-  
compose.yml  
- Part  
D/Task  
5/Dock-  
erfile -  
Part  
D/Task  
5/app.js  
###  
docker-  
compose.yml  
“‘bash  
ver-  
sion:  
‘3.8’

---

```

services:
db:
im-
age:
mysql:5.7
con-
tainer__name:
mysql-
persistent
envi-
ron-
ment:
MYSQL_ROOT_PASSWORD:
lab-
pass-
word
MYSQL_DATABASE:
inven-
tory
vol-
umes:
-
db__data:/var/lib/mysql
restart:
al-
ways
volumes:
db__data:
““
###
app.js
“‘bash
const
http
= re-
quire('http');
http.createServer((req,
res)
=> {
res.end('Docker
Com-
pose
Build
Lab');
}).lis-
ten(3000);
###
Dockerfilebash
FROM
node:18-
alpine
WORKDIR
/app

```



---

COPY  
app.js  
.  
EXPOSE  
3000  
CMD  
["node",  
"app.js"]  
###  
Im-  
age  
Gallery  
The  
fol-  
low-  
ing  
im-  
ages  
docu-  
ment  
the  
lab  
execu-  
tion  
and  
re-  
sults:  
  
###  
Notes

---

- The root docker-compose.yml in lab/exp6 is configured to build a Node app from a local Dockerfile.

- If that Dockerfile is not present in the root folder, use the Part C/Task 4 or Part D/Task 5 examples as working references for custom builds.

- Use docker compose ps, docker ps, ~~any~~ docker logs for

---

#  
Lab  
7:  
CI/CD  
Pipeline  
with  
Jenk-  
ins,  
GitHub,  
and  
Docker  
###  
ALL  
THE  
SCREEN-  
SHOTS  
ARE  
AT  
THE  
END  
OF  
THE  
EX-  
PER-  
I-  
MENT  
##  
Ob-  
jec-  
tive

---

Build  
a  
com-  
plete  
CI/CD  
pipeline  
that  
auto-  
mati-  
cally  
builds  
Docker  
im-  
ages  
from  
GitHub  
code  
and  
pushes  
them  
to  
Docker  
Hub  
when-  
ever  
code  
is  
com-  
mit-  
ted.

---

## 24.4 Architecture

GitHub Repository  
↓ (webhook on push)  
Jenkins Server  
↓ (checkout & build)  
Build Docker Image  
↓ (push)  
Docker Hub Registry

---

## 24.5 Process Done

### 24.5.1 Step 1: Jenkins Setup

Installed Jenkins using Docker with Docker socket access:

```
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
-v /var/run/docker.sock:/var/run/docker.sock \
-v /usr/bin/docker:/usr/bin/docker \
jenkins/jenkins:lts
```

Jenkins running at <http://localhost:8080>

---

### 24.5.2 Step 2: Installed Jenkins Plugins

Required plugins installed: - GitHub Plugin (webhook triggers) - Docker Pipeline (build Docker images) - Credentials Binding Plugin (secure credentials) - Pipeline Plugin (declarative pipelines)

All plugins active

---

### 24.5.3 Step 3: Added Docker Hub Credentials

Stored credentials securely in Jenkins: - **Manage Jenkins** → **Manage Credentials** → **Global credentials**  
- Created credential with ID: `docker-hub-creds` - Used Personal Access Token (not password for security)

Credentials stored

---

### 24.5.4 Step 4: Created GitHub Repository

Repository structure:

```
my-app/
 Dockerfile (multi-stage build)
 Jenkinsfile (pipeline configuration)
 src/ (application code)
 package.json (dependencies)
```

Repository ready

---

### 24.5.5 Step 5: Created Pipeline Job

Configured Jenkins pipeline: - Job name: `my-app-pipeline` - Type: Pipeline (Declarative) - SCM: Git - Script path: `Jenkinsfile`

Job created and configured

---

### 24.5.6 Step 6: Configured GitHub Webhook

Set up automatic trigger: - Webhook URL: `http://YOUR_JENKINS_IP:8080/github-webhook/` - Content type: JSON - Events: Push and Pull Request

Webhook verified (green checkmark)

---

## 24.6 Jenkinsfile Pipeline

```
pipeline {
 agent docker

 stages {
 stage('Checkout') {
 steps { checkout scm }
 }
 }
}
```

```

stage('Build & Test') {
 steps { sh 'npm install && npm test' }
}

stage('Build Docker Image') {
 steps { sh 'docker build -t myapp:${BUILD_NUMBER} .' }
}

stage('Push to Docker Hub') {
 steps {
 withCredentials([usernamePassword(credentialsId: 'docker-hub-creds', usernameVariable:
 sh '''
 echo $PASS | docker login -u $USER --password-stdin
 docker tag myapp:${BUILD_NUMBER} $USER/myapp:latest
 docker push $USER/myapp:latest
 ''')
]
)
}
}
}
}
}

```

---

## 24.7 How It Works

When code is pushed to GitHub:

1. **GitHub webhook** sends notification to Jenkins
2. **Jenkins** automatically checks out code
3. **Build stage** - compiles and runs tests
4. **Docker stage** - builds Docker image
5. **Push stage** - logs into Docker Hub and pushes image
6. **Cleanup stage** - removes old images

All **automatic** - no manual steps needed!

---

## 24.8 Testing

### 24.8.1 Manual Build

Jenkins Dashboard → my-app-pipeline → Build Now

Build succeeds → Image pushed to Docker Hub

### 24.8.2 Automated Trigger

```

git add .
git commit -m "Test trigger"
git push origin main

```

Webhook triggers → Build starts → Image pushed automatically

---

## 24.9 Quick Commands

*# Check Jenkins*

```
curl http://localhost:8080
```

*# View logs*

```
docker logs -f jenkins
```

*# Manually build*

```
curl -X POST http://localhost:8080/job/my-app-pipeline/build
```

*# View image*

```
docker pull YOUR_USERNAME/myapp:latest
```

---

## 24.10 Key Features

Automated builds on every GitHub push  
Secure credentials stored in Jenkins  
Docker images built automatically  
Images pushed to Docker Hub  
Webhook triggers instant builds  
Build logs visible for debugging  
Scalable for multiple projects

---

## 24.11 Conclusion

Complete CI/CD pipeline is now functional. Any code pushed to GitHub automatically: 1. Triggers Jenkins build 2. Builds Docker image 3. Pushes to Docker Hub

No manual intervention required!



---

## ## Screen- shots



---

#  
Lab  
9:  
Docker  
Con-  
tainer  
with  
SSH  
Con-  
figu-  
ra-  
tion  
and  
Ansi-  
ble  
De-  
ploy-  
ment  
##  
Ob-  
jec-  
tive  
To  
cre-  
ate a  
Docker  
con-  
tainer  
with  
SSH  
server  
capa-  
bili-  
ties  
and  
con-  
figure  
it  
using  
Ansi-  
ble  
for  
infras-  
truc-  
ture  
au-  
toma-  
tion  
and  
server  
man-  
age-  
ment.

---

##  
Tools  
and  
Tech-  
nolo-  
gies  
Used -  
**Docker:**  
For  
con-  
tainer-  
iza-  
tion -  
**An-  
sible:**  
For  
con-  
figu-  
ra-  
tion  
man-  
age-  
ment  
and  
au-  
toma-  
tion -  
**Ubuntu:**  
Base  
oper-  
ating  
sys-  
tem -  
**SSH:**  
Se-  
cure  
Shell  
for re-  
mote  
ac-  
cess -  
**Python:**  
Script-  
ing  
lan-  
guage

---

## 24.12 Experiment Steps

### 24.12.1 Step 1: Build the Docker Image from Dockerfile

**Description:** Create a Docker image that includes SSH server, Python 3, and SSH key authentication.

**Command:**

```
docker build -t ubuntu-server .
```

**Expected Output:** Image successfully built Step 1

---

### 24.12.2 Step 2: Run the Docker Container

**Description:** Launch the Docker container with SSH port mapping and resource constraints.

**Command:**

```
docker run -d --rm -p 2222:22 -p 8221:8221 --name ssh-test-server ubuntu-server
```

**Parameters Explained:** - -d: Run in detached mode (background) - --rm: Automatically remove container when it exits - -p 2222:22: Map host port 2222 to container SSH port 22 - -p 8221:8221: Map additional port for application services - --name ssh-test-server: Container name for easy reference - ubuntu-server: Image name to run

**Expected Output:** Container ID returned Step 2

---

### 24.12.3 Step 3: Verify Container Status

**Description:** Check if the container is running and ports are properly mapped.

**Command:**

```
docker ps
```

**Expected Output:** Running container with port mappings visible Step 3

---

### 24.12.4 Step 4: Test SSH Connection to Container

**Description:** Establish SSH connection to the running container using port 2222.

**Command:**

```
ssh -p 2222 root@localhost
```

**Expected Output:** SSH connection prompt (you may need to accept key fingerprint)

```
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:WwmJqPAsxsn8ARhMA0J1M1e+wIqqVh262gnQ6B/Do5s.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

Step 4

Figure 26: Step 4

---

### 24.12.5 Step 5: Configure Ansible Inventory

**Description:** Set up the inventory file with container host information for Ansible to connect to.

**Inventory File Content (inventory.ini):**

```
[servers]
localhost:2222
```

```
[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
```

**Key Configuration:** - Define server group for organization - Specify SSH user and private key path - Set Python interpreter path

Step 5

Figure 27: Step 5

---

### 24.12.6 Step 6: Run Ansible Playbook on Container

**Description:** Execute Ansible playbook to perform automated configuration tasks on the container.

**Command:**

```
ansible-playbook -i inventory.ini playbook1.yml
```

**Playbook Tasks** (from `playbook1.yml`): 1. Update apt package index 2. Install Python 3 (latest available) 3. Create test configuration file with server metadata 4. Execute system information commands (`uname`, `df`) 5. Display results for verification

**Expected Output:** Playbook execution report with task results Step 6

---

### 24.12.7 Step 7: Verify Automation Results

**Description:** Check if all Ansible tasks completed successfully and verify the configuration applied.

**Verification Commands:**

```
Check test file was created
ssh -p 2222 root@localhost "cat /root/test_file.txt"
```

```
Verify Python installation
ssh -p 2222 root@localhost "python3 --version"
```

```
Check system packages updated
ssh -p 2222 root@localhost "apt list --upgradable"
```

**Expected Output:** Confirmation that all tasks executed successfully Step 7

---

## 24.13 Cleanup Commands

Stop and remove the container:

```
docker stop ssh-test-server
docker rm ssh-test-server
```

Remove the Docker image:

```
docker rmi ubuntu-server
```

---

## 24.14 Key Learnings

1. **Docker SSH Configuration:** Setting up SSH in Docker requires proper key permissions (600 for private keys, 644 for public keys).
  2. **Port Mapping:** Docker allows flexible port mapping for accessing services running in containers.
  3. **Ansible Automation:** Ansible simplifies infrastructure management by allowing declarative configuration of multiple servers.
  4. **Infrastructure as Code:** Using Ansible playbooks provides reproducible, version-controlled infrastructure setup.
  5. **Security Considerations:** Using SSH key-based authentication is more secure than password-based authentication.
- 

## 24.15 Troubleshooting

### 24.15.1 Issue: Permission Denied (publickey)

**Solution:** Ensure SSH public key is in `/root/.ssh/authorized_keys` with correct permissions (644).

### 24.15.2 Issue: Connection Refused

**Solution:** Verify container is running with `docker ps` and SSH service is active inside container.

### 24.15.3 Issue: Ansible Connection Failed

**Solution:** Check `inventory.ini` file paths and ensure private key file exists at specified location.

---

## 24.16 Conclusion

This lab successfully demonstrates the integration of Docker containers with Ansible for automated infrastructure management. The combination allows for scalable, repeatable deployments across multiple containerized environments, making it a powerful approach for DevOps workflows.

# 25 Lab 10: CI/CD Pipeline with Jenkins and SonarQube

## 25.1 Introduction

This lab demonstrates how to automate the build, test, and code quality analysis of a Java application using Jenkins and SonarQube. By the end, you will have a working pipeline that checks your code for errors and quality issues every time you push changes.

## 25.2 Prerequisites

- Docker and Docker Compose installed
- Basic knowledge of Java and Maven
- Familiarity with Jenkins and SonarQube concepts

## 25.3 Project Structure

```
Lab10/
 docker-compose.yml
 jenkinsfile
 readme.md
 sample-java-app/
```

```
pom.xml
src/
 main/
 java/
...

```

## 25.4 Step-by-Step Instructions

### 1. Clone the repository

```
git clone <repo-url>
cd Lab10
```

### 2. Start Jenkins and SonarQube using Docker Compose

```
docker-compose up -d
```

### 3. Access Jenkins

- Open your browser and go to <http://localhost:8080>
- Complete the initial setup and install recommended plugins

### 4. Access SonarQube

- Go to <http://localhost:9000>
- Log in with default credentials (admin/admin)

### 5. Configure Jenkins Pipeline

- Create a new pipeline job
- Use the provided `jenkinsfile` for pipeline configuration

### 6. Run the Pipeline

- Trigger a build and observe the stages: build, test, and code analysis

## 25.5 Jenkins Pipeline Overview

The Jenkins pipeline defined in `jenkinsfile` automates the following tasks: - Build the Java application using Maven - Run unit tests - Analyze code quality with SonarQube - Archive build artifacts

## 25.6 SonarQube Integration

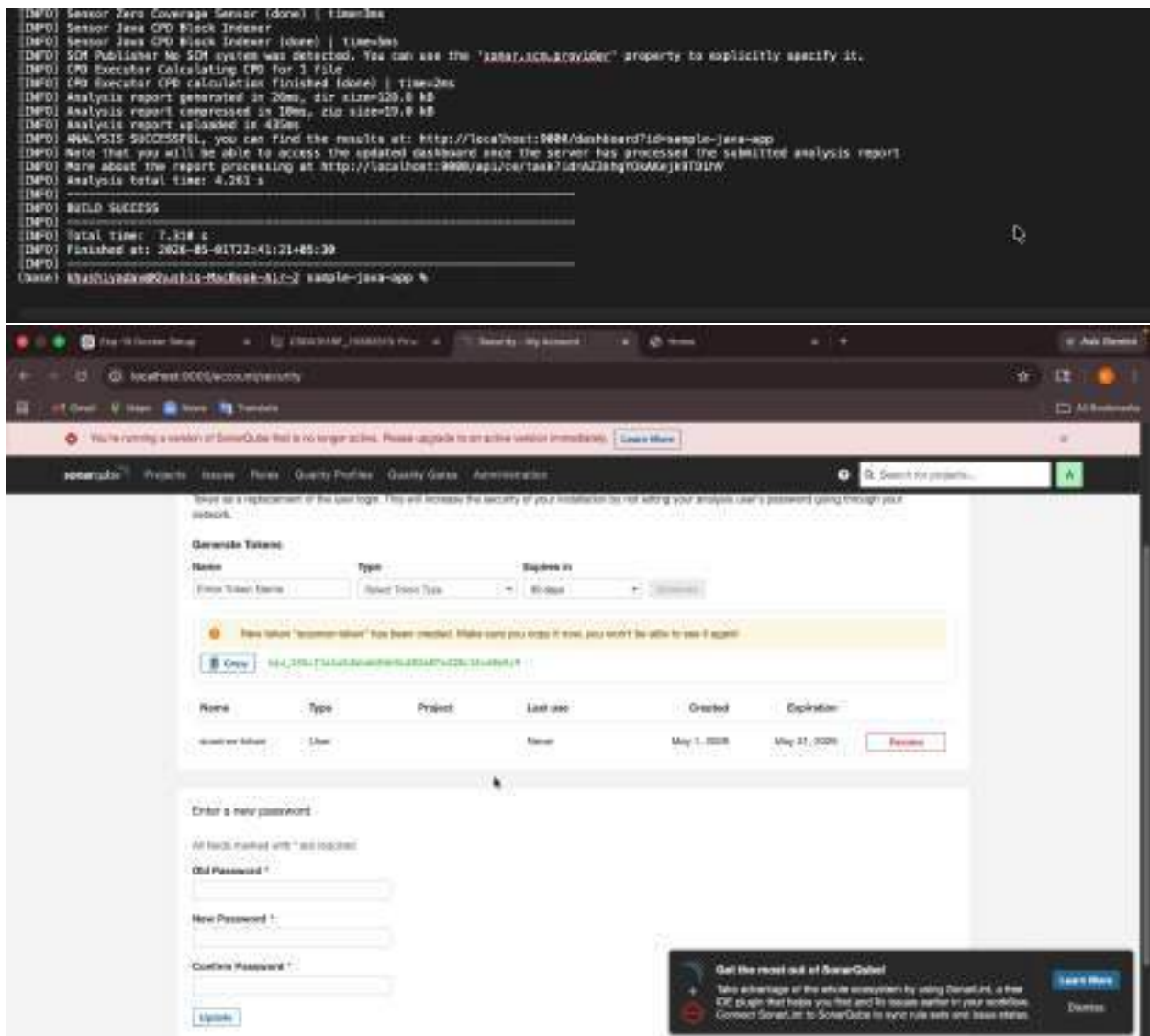
SonarQube is used to analyze the code for bugs, vulnerabilities, and code smells. The pipeline sends analysis results to SonarQube, which you can review in the SonarQube dashboard.

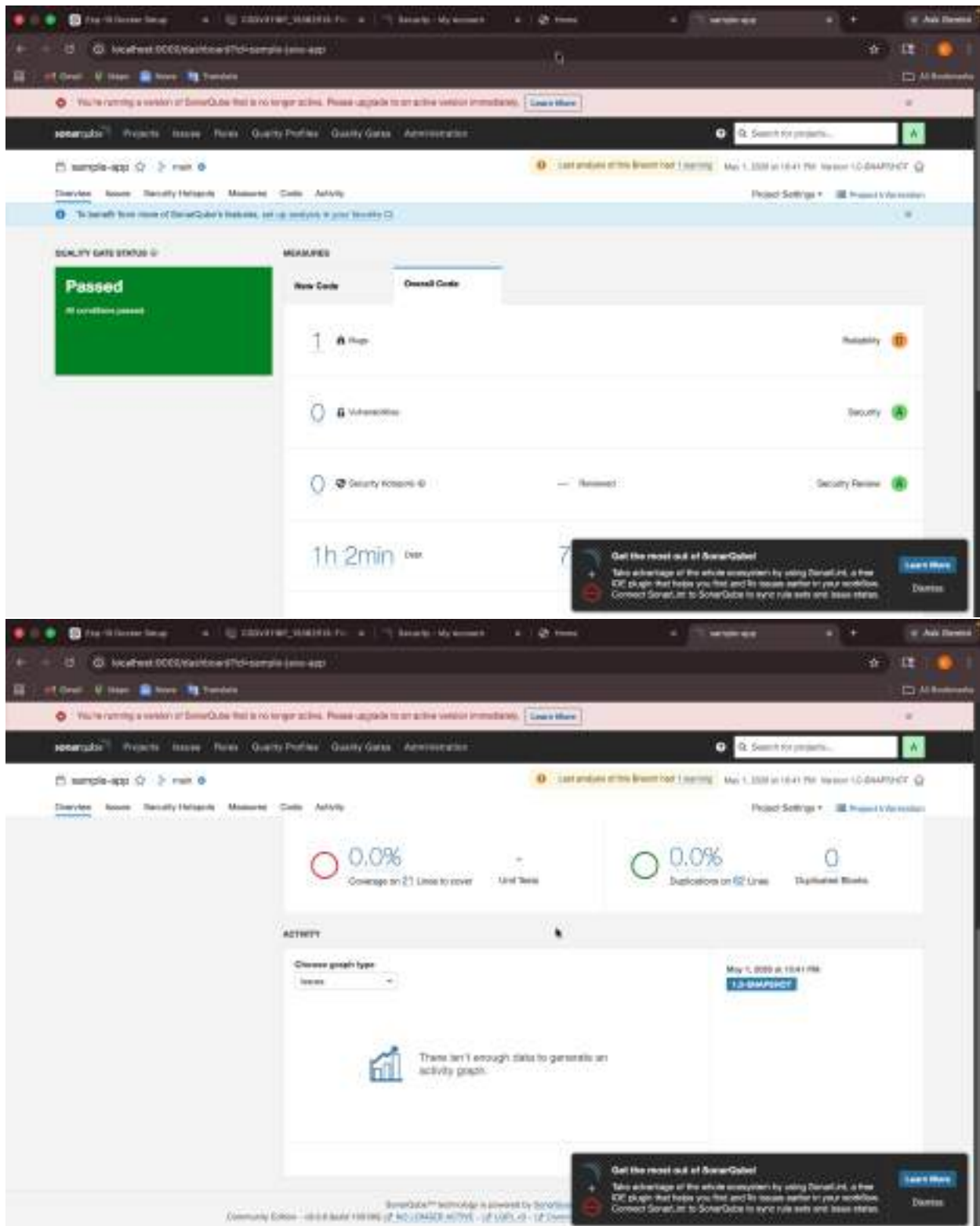
## 25.7 Screenshots

Below are screenshots showing the setup and results at each stage:









## 25.8 Troubleshooting

- If Jenkins or SonarQube do not start, check Docker logs for errors.
- Ensure ports 8080 (Jenkins) and 9000 (SonarQube) are not in use by other applications.

- If SonarQube analysis fails, verify the SonarQube server URL and authentication token in Jenkins.

## 25.9 Conclusion

You have now set up a basic CI/CD pipeline with Jenkins and SonarQube for a Java application. This setup can be extended to include more advanced features like deployment, notifications, and additional quality gates.

---

Feel free to experiment and modify the pipeline to suit your project needs. Happy coding!

# 26 Lab 11: Deploying WordPress with Docker Swarm

## 26.1 Introduction

This experiment demonstrates how to deploy a WordPress website using Docker Swarm for container orchestration. You will set up a multi-container application with WordPress and MySQL, leveraging Docker Swarm's clustering and scaling features.

---

## 26.2 Objectives

- Initialize a Docker Swarm cluster
- Deploy WordPress and MySQL as services
- Use persistent storage with Docker volumes
- Access the WordPress site from your browser

---

## 26.3 Prerequisites

- Docker and Docker Compose installed
- Basic knowledge of Docker concepts
- Access to a terminal with administrative privileges

---

## 26.4 Steps

### 26.4.1 1. Initialize Docker Swarm

If your system has multiple network interfaces, specify the IP address to advertise:

```
docker swarm init --advertise-addr <YOUR_IP_ADDRESS>
```

Replace <YOUR\_IP\_ADDRESS> with your main network IP (e.g., 172.30.10.156).

### 26.4.2 2. Deploy the Stack

Use the provided `docker-compose.yml` to deploy the stack as a Swarm stack:

```
docker stack deploy -c docker-compose.yml wordpress_stack
```

### 26.4.3 3. Check Service Status

List the running services:

```
docker service ls
```

#### 26.4.4 4. Access WordPress

- Open your browser and go to `http://<YOUR_IP_ADDRESS>:8080`
- Complete the WordPress setup wizard

#### 26.4.5 5. Scaling Services (Optional)

To scale the WordPress service:

```
docker service scale wordpress_stack_wordpress=2
```

---

### 26.5 docker-compose.yml Overview

- **db**: Runs MySQL with persistent storage
  - **wordpress**: Runs the WordPress application, depends on the database
  - **Volumes**: `db_data` and `wp_data` ensure data persists across restarts
- 

### 26.6 Troubleshooting

- If you see an error about multiple IP addresses, use `--advertise-addr` as shown above.
  - Ensure ports 8080 (WordPress) and 3306 (MySQL) are not blocked or in use.
  - Use `docker service ps <SERVICE_NAME>` to check for errors in service startup.
- 

### 26.7 Conclusion

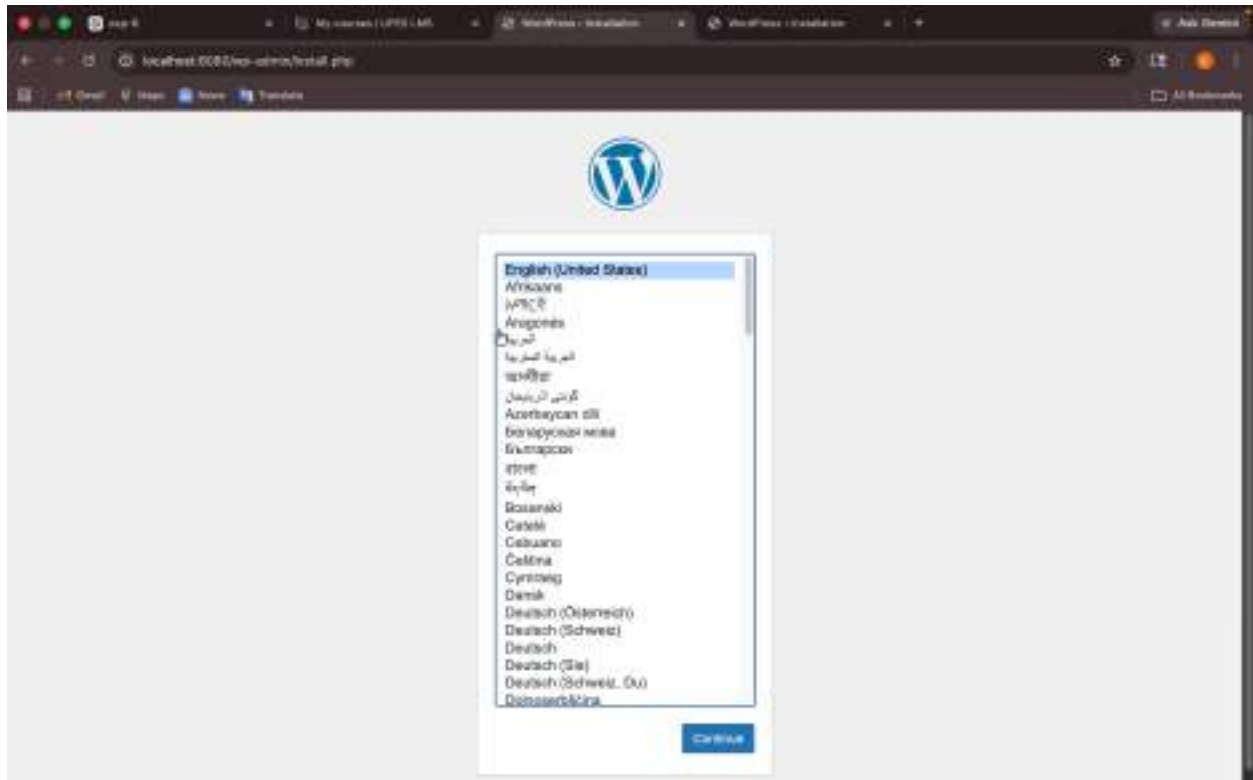
You have successfully deployed a WordPress site using Docker Swarm. This setup can be extended to include more services, enable scaling, and provide high availability.





## 26.8 Screenshots

[illegible]



Feel free to experiment with scaling and updating services to explore more Docker Swarm features!

## 27 Kubernetes Lab Experiment – Deploying WordPress using kubectl

## 27.1 Overview

This experiment demonstrates the fundamentals of Kubernetes by deploying a real-world application (WordPress) using core Kubernetes objects like Deployments and Services. It also explores scaling and self-healing capabilities of Kubernetes in a cluster created using `kubeadm`.

## 27.2 Objectives

- Understand basic Kubernetes architecture and components
- Set up a Kubernetes cluster using `kubeadm`
- Deploy a WordPress application using a Deployment
- Expose the application using a Service
- Perform scaling operations
- Observe self-healing behavior of Kubernetes

### 27.3 Prerequisites

- Linux environment (Ubuntu / WSL / Virtual Machine)
- Container runtime (Docker / containerd)



- Installed tools:
    - kubectl
    - kubeadm
    - kubelet
  - Minimum 2 GB RAM recommended
  - Internet connectivity
- 

## 27.4 Cluster Setup using kubeadm

### 27.4.1 Step 1: Initialize the Kubernetes Cluster

```
sudo kubeadm init
```

---

### 27.4.2 Step 2: Configure kubectl for the user

```
mkdir -p $HOME/.kube
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

---

### 27.4.3 Step 3: Install Network Plugin (Calico)

```
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
```

---

## 27.5 Application Deployment (WordPress)

### 27.5.1 Step 4: Deploy WordPress

```
kubectl apply -f wordpress-deployment.yaml
```

---

### 27.5.2 Step 5: Expose WordPress Service

```
kubectl apply -f wordpress-service.yaml
```

---

## 27.6 Verification Commands

### 27.6.1 Check Nodes

```
kubectl get nodes
```

### 27.6.2 Check Pods

```
kubectl get pods
```

### 27.6.3 Check Services

```
kubectl get svc
```

---

## 27.7 Scaling the Application

Increase the number of replicas:

```
kubect1 scale deployment wordpress --replicas=3
```

Verify scaling:

```
kubect1 get pods
```

---

## 27.8 Self-Healing Test

Delete a running pod:

```
kubect1 delete pod <pod-name>
```

Kubernetes will automatically recreate the pod to maintain the desired state.

---

## 27.9 Observations

- Deployments manage pod lifecycle automatically
  - Services provide stable network access to applications
  - Kubernetes maintains the desired state continuously
  - Pods are recreated automatically when they fail (self-healing)
  - Scaling can be done dynamically without downtime
- 

## 27.10 Issues Faced During Experiment

- Kubernetes cluster initialization failure using kubeadm
  - API Server connection refused errors (`connection refused 6443`)
  - Problems due to WSL environment limitations
  - Container runtime and system configuration issues
- 

## 27.11 Conclusion

In this experiment, we successfully:

- Understood Kubernetes architecture and working
- Deployed a real-world application (WordPress)
- Performed scaling operations
- Tested self-healing capability
- Explored cluster setup using kubeadm ## Screenshots screenshot screenshot screenshot screenshot